ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP LỚN

KỸ THUẬT DỮ LIỆU (CO5173)

Data Engineering

Đề tài: Project

NỀN TẢNG KỸ THUẬT DỮ LIỆU CHO HỆ THỐNG GIÁO DỤC TRỰC TUYẾN QUY MÔ LỚN

Learning Analytics, gợi ý học tập và cảnh báo sớm nguy cơ bỏ học

---

Giảng viên hướng dẫn	PGS. TS. Võ Thị Ngọc Châu
Lớp / Nhóm	[Lớp] / [Nhóm]
Học kỳ	Học kỳ 1, năm học 2026–2027

Học viên thực hiện

Họ và tên	**MSHV**	**Phần việc chính**
[Phạm Duy Anh]	[2310139]	[BR4, BR5]
[Họ và tên thành viên 2]	[MSHV]	[Phần việc]
[Họ và tên thành viên 3]	[MSHV]	[Phần việc]
[Họ và tên thành viên 4]	[MSHV]	[Phần việc]

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

Tóm tắt **9**

Danh mục thuật ngữ và từ viết tắt **11**

1 Bài toán, các bên liên quan và yêu cầu **13**

1.1 Giới thiệu 13

1.1.1 Bối cảnh 13

1.1.2 Phát biểu bài toán 13

1.1.3 Mục tiêu và phạm vi 14

1.1.4 Đóng góp chính 14

1.1.5 Phương pháp thực hiện 15

1.1.6 Cấu trúc báo cáo 15

1.2 Phân tích yêu cầu và nguồn dữ liệu 15

1.2.1 Các bên liên quan và yêu cầu nghiệp vụ 15

1.2.2 Nguồn dữ liệu 17

1.2.3 Yêu cầu phi chức năng 18

1.2.4 Ánh xạ yêu cầu vào pipeline 18

2 Kiến trúc và lựa chọn công nghệ **20**

2.1 Kiến trúc tổng thể 20

2.1.1 Sáu tầng của kiến trúc 20

2.1.2 Lựa chọn công nghệ 21

2.1.3 Kiến trúc phần mềm: DDD và hexagonal 23

2.1.4 Ngữ nghĩa stream processing 23

2.1.5 Các quyết định kiến trúc 24
2.1.6 Điều chỉnh so với kiến trúc tham chiếu 25

3 Lakehouse và data warehouse 27
3.1 Tổng quan lakehouse 27
3.1.1 Kiến trúc medallion 27
3.1.2 Chuẩn hoá telemetry và schema evolution 28
3.1.3 Pseudonymisation và phân quyền 29
3.1.4 Star schema Kimball ở lớp gold 29
3.1.5 Data mart theo yêu cầu nghiệp vụ 31
3.1.6 Thiết kế vật lý 32
3.1.7 Quality gate 32
3.1.8 Điều phối, late data và rerun 33

4 Hiện thực BR1–BR8 và ứng dụng 36
4.1 Thiết kế và hiện thực theo từng yêu cầu 36
4.1.1 BR1 — Pipeline thông báo hướng sự kiện 36
4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo 37
4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết 38
4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường 39
4.1.5 BR5 — Cảnh báo sớm nguy cơ dropout 41
4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức 42
4.1.7 BR7 — Heatmap tương tác video 42
4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác 43
4.1.8.1 Đồng bộ cache và search index 43
4.1.8.2 Xu hướng tương tác 44
4.2 Giao diện người dùng 44
4.2.1 Quy trình thiết kế 44
4.2.2 Đáp ứng yêu cầu về giao diện 45
4.2.3 Design system 47
4.2.4 Kiến trúc frontend 47

4.2.5 Các màn hình chính 48
4.2.6 Kiểm thử giao diện 53

5 Kiểm thử và đánh giá **55**
5.1 Chiến lược kiểm thử 55
5.2 Kiểm chứng nghiệp vụ 56
5.3 Môi trường đo 58
5.4 Kết quả hiệu năng 60
5.4.1 BR1–BR3: end-to-end latency 60
5.4.2 BR4: tổng hợp SQL so với Python 61
5.4.3 BR5: end-to-end latency và throughput của window 62
5.4.4 BR6–BR8: serving mart 63
5.4.5 BR8: từ xuất bản đến khi tìm kiếm được 63
5.4.6 BR7/BR8: overlay thời gian thực 64
5.4.7 Thời gian build data warehouse 64
5.5 Debezium và polling: negative-control benchmark 65
5.5.1 Thiết lập 65
5.5.2 Kết quả 66
5.5.3 Ý nghĩa với bài toán dữ liệu lớn 66
5.5.4 Giới hạn của phép đo 67
5.6 Thảo luận 67

6 Hạn chế, hướng phát triển và kết luận **69**
6.1 Hạn chế hiện tại 69
6.2 Hướng phát triển 70
6.3 Kết luận 70

Tài liệu tham khảo **72**

A Hướng dẫn chạy và tái lập kết quả **74**
A.1 Yêu cầu 74
A.2 Khởi động 74

A.3 Kiểm thử 76
A.4 Đo hiệu năng 76
A.5 Đo Debezium và polling 77
A.6 Biên dịch báo cáo 78

B Tóm tắt API **79**

Danh sách hình vẽ

2.1 Kiến trúc tổng thể của nền tảng Project 21

3.1 Các lớp của lakehouse và tần suất cập nhật 27

4.1 Trích tập rule thông báo 37
4.2 Đường CDC thật của BR4 và BR5 40
4.3 Các window feature tại evaluation time T (BR5) 41
4.4 Đồng bộ nội dung BR8 dựa trên CDC 43
4.5 Mockup: Dashboard người học với thẻ khoá học (BR2), thông báo (BR1) và phản hồi gần đây (BR3) 45
4.6 Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff 49
4.7 Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20% 49
4.8 Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video “Joins” (390 giây), kèm các thời điểm cần xem lại 50
4.9 Learners to support (BR5): queue rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15 50
4.10 Organisation progress (BR6): k-anonymity với $k = 5$ 51
4.11 Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được 52
4.12 Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu . . . 52
4.13 Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang 53

5.1 BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log) . . 62

5.2 Latency serving các mart BR6–BR8 (30 request mỗi trường hợp) 63
5.3 BR8: latency từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp . . . 63
5.4 Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần) . . 64

Danh sách bảng

1.1 Yêu cầu nghiệp vụ và nhu cầu data engineering 16
1.2 Bốn nguồn dữ liệu và cách tiếp nhận 17
1.3 Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime 19

2.1 Công nghệ theo vai trò 22
2.2 Các lớp trong một bounded context 23
2.3 Tóm tắt các quyết định kiến trúc (ADR) 24

3.1 Data contract của từng lớp 28
3.2 Phân quyền truy cập data warehouse qua Trino 29
3.3 Bus matrix của lớp gold (D: vai trò ngày, •: khoá dimension) 30
3.4 Các data mart và feature store 31
3.5 Các quality gate 33
3.6 Các DAG Airflow 34
3.7 Cơ chế xử lý late data, dữ liệu trùng lặp và out-of-order 34

4.1 Các tier của rule gợi ý (BR2) 38
4.2 Rule phát hiện bất thường của BR4 (có thể cấu hình) 40
4.3 Các tín hiệu rủi ro và điểm số (BR5) 42
4.4 Trọng số hoạt động trong engagement score (`EngagementWeights`) 44
4.5 Đặc điểm người dùng và lựa chọn thiết kế tương ứng 46
4.6 Workspace, route và trang 47

5.1 Các tầng kiểm thử 56
5.2 Các kịch bản kiểm chứng nghiệp vụ và kết quả 57

5.3 Container chạy trong từng phép đo (stack đầy đủ: 30 container) 59
5.4 Môi trường đo hiệu năng 60
5.5 Latency BR1–BR3 (ms) 61
5.6 BR5: chặng Kafka → Spark → window đã lưu (trung bình 3 lần) 62
5.7 Latency từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước) 64
5.8 Debezium và polling trên cùng 100.000 dòng 66

B.1 Các endpoint chính 79

Tóm tắt

Báo cáo trình bày thiết kế và hiện thực **Project**, một nền tảng data engineering cho hệ thống giáo dục trực tuyến quy mô lớn. Nền tảng tiếp nhận bốn nguồn dữ liệu vốn rời rạc — telemetry clickstream từ web và mobile, CDC từ PostgreSQL, kho học liệu phi cấu trúc và log của sandbox chấm bài — rồi chuyển hoá chúng thành một data warehouse đáp ứng tám yêu cầu nghiệp vụ (BR1–BR8) của bốn nhóm người dùng: người học, giảng viên, cố vấn học tập và quản trị viên tổ chức.

Về kiến trúc, Kafka và Debezium đảm nhận tầng ingestion, trong đó Kafka là event bus; Spark (batch và Structured Streaming) là engine xử lý chính; Delta Lake trên MinIO tổ chức dữ liệu theo mô hình medallion bronze–silver–gold, trong đó lớp gold là data warehouse theo star schema Kimball; Hive Metastore làm catalog và Trino phục vụ SQL; Redis đảm bảo serving latency thấp, OpenSearch cung cấp tìm kiếm lai. Mã nguồn được chia thành bảy bounded context theo DDD và kiến trúc hexagonal, nhờ đó cùng một use case có thể chạy trên nhiều engine mà vẫn kiểm thử được độc lập với engine. Mười sáu ADR ghi lại các quyết định thiết kế, kể cả những điều chỉnh so với kiến trúc tham chiếu. BR1–BR3 chạy end-to-end trên Kafka, Redis, PostgreSQL và Delta; BR4–BR8 chạy trên hạ tầng lakehouse.

Kết quả đo trên máy cá nhân với dữ liệu tổng hợp được tóm tắt như sau:

- **BR1–BR3**: median 12,4 ms từ lúc phát sự kiện đến khi thông báo đọc được, 2,2 ms cho API gợi ý, 14,2 ms để nhận 202 và 71,0 ms từ lúc nộp bài đến khi chấm xong.
- **BR5**: từ thay đổi CDC đến cảnh báo dropout mất khoảng 10,2 giây (kịch bản chấp nhận, không đo lại).
- **BR8**: nội dung xuất bản được index sau median 501 ms.
- **BR7–BR8**: overlay hiển thị sau khoảng 6 giây và giữ ổn định khi tải tăng 15 lần.
- **BR6–BR8**: mart truy vấn qua Trino mất 81–172 ms; qua API có cache còn 2,6–6,0 ms.
- **Data warehouse**: 43 model được build trong khoảng 2 phút (121–130 giây) và đạt toàn bộ quality gate.

Hệ thống có 379 unit test, 1 integration test end-to-end, 12 browser test cùng các kịch bản kiểm

chứng nghiệp vụ. Báo cáo cũng trình bày giao diện web, những hạn chế còn lại và hướng phát triển.

Từ khoá: data engineering, lakehouse, Delta Lake, Kafka, Debezium, CDC, Spark Structured Streaming, learning analytics.

Danh mục thuật ngữ và từ viết tắt

Thuật ngữ	Giải thích
ADR	*Architecture Decision Record*: tài liệu ghi lại một quyết định kiến trúc cùng bối cảnh, các phương án bị loại và hệ quả.
BR	*Business Requirement*: yêu cầu nghiệp vụ (BR1–BR8) do nhóm sinh viên lựa chọn.
Bronze / Silver / Gold	Ba lớp của mô hình medallion: dữ liệu thô, dữ liệu đã làm sạch và pseudonymise, dữ liệu được mô hình hoá phục vụ phân tích.
CDC	*Change Data Capture*: bắt các thay đổi cấp dòng từ transaction log (WAL) của cơ sở dữ liệu.
CDF	*Change Data Feed*: tính năng của Delta Lake ghi lại các thay đổi cấp dòng của một bảng.
DDD	*Domain-Driven Design*: thiết kế hướng miền nghiệp vụ, chia hệ thống thành các bounded context.
DLQ	*Dead-Letter Queue*: nơi lưu các sự kiện bị từ chối kèm lý do để xử lý lại.
Event time	Thời điểm sự kiện xảy ra tại nguồn, khác với thời điểm hệ thống tiếp nhận sự kiện đó.
Exactly-once	Bảo đảm mỗi bản ghi chỉ tác động lên kết quả đúng một lần, kể cả khi có lỗi hoặc gửi lại.
Feature store	Kho feature dùng chung cho huấn luyện (offline) và phục vụ trực tuyến (online).
Hexagonal	Kiến trúc lục giác (ports and adapters): lõi nghiệp vụ chỉ phụ thuộc vào các cổng trừu tượng.
HMS	Hive Metastore: dịch vụ catalog dùng chung cho các bảng Delta.
Idempotent	Chạy lại nhiều lần vẫn cho cùng kết quả như khi chạy một lần.

Thuật ngữ	Giải thích
k-anonymity	Nguyên tắc không công bố số liệu của nhóm có ít hơn k cá thể; trong báo cáo này $k = 5$.
LSN	*Log Sequence Number*: vị trí trong WAL của PostgreSQL, dùng để sắp thứ tự các thay đổi.
Mart	Data mart: bảng tổng hợp phục vụ một yêu cầu nghiệp vụ cụ thể.
Medallion	Mô hình tổ chức lakehouse theo các lớp có mức độ tinh lọc tăng dần.
OLTP	*Online Transaction Processing*: cơ sở dữ liệu giao dịch của hệ thống quản lý học tập (LMS).
Overlay	Lớp dữ liệu bổ sung, cập nhật liên tục, được ghép với mart theo ngày để phản ánh hoạt động trong ngày.
PII	*Personally Identifiable Information*: thông tin định danh cá nhân.
Pseudonym	Giá trị băm có khoá (HMAC) thay cho định danh thật.
SCD	*Slowly Changing Dimension*: chiến lược lưu lịch sử thay đổi của bảng dimension (loại 1 ghi đè, loại 2 lưu phiên bản).
Watermark	Ngưỡng thời gian mà engine streaming dùng để xác định đã nhận đủ dữ liệu tính đến một mốc event time.

Chương 1

Bài toán, các bên liên quan và yêu cầu

1.1 Giới thiệu

1.1.1 Bối cảnh

Một nền tảng giáo dục trực tuyến quy mô lớn sinh ra dữ liệu liên tục và rất đa dạng. Mỗi thao tác của người học — mở bài giảng, tua video, nộp bài lập trình, tham gia thảo luận — đều để lại dấu vết. LMS ghi nhận tiến độ, bài nộp và điểm số; kho học liệu chứa transcript, slide và PDF được cập nhật theo phiên bản; sandbox chấm bài sinh ra log thực thi chi tiết. Khối dữ liệu này có giá trị lớn đối với learning analytics: giúp giảng viên phát hiện bài tập có vấn đề, giúp cố vấn nhận diện sớm nguy cơ dropout và giúp người học nhận phản hồi kịp thời.

Tuy nhiên, các nguồn dữ liệu nói trên rời rạc và khác nhau về định dạng, tốc độ lẫn độ tin cậy. Telemetry từ thiết bị di động có thể đến muộn nhiều ngày hoặc out-of-order; thay đổi trong cơ sở dữ liệu giao dịch phải được bắt đúng thứ tự mà không gây tải lên hệ thống production; tài liệu học tập cần được chunk và quản lý phiên bản; dữ liệu cá nhân phải được bảo vệ, nhất là trong báo cáo gửi cho tổ chức đối tác (B2B). Đây là bài toán data engineering điển hình, đòi hỏi kết hợp stream processing, batch processing, lưu trữ phân tích và serving latency thấp [1].

1.1.2 Phát biểu bài toán

Project hướng tới xây dựng một nền tảng dữ liệu biến bốn nguồn thô thành data warehouse phục vụ ba mục tiêu: phân tích học tập, gợi ý thích ứng và cảnh báo sớm nguy cơ dropout. Từ yêu cầu thiết kế một hệ thống dữ liệu trong giáo dục, nhóm sinh viên lựa chọn tám yêu cầu data engineering (BR1–BR8) cho bốn nhóm người dùng và chọn kiến trúc tham chiếu gồm Kafka và Debezium cho ingestion, Flink cho streaming, Spark cho batch, Delta Lake cho lakehouse và

Redis cho serving layer. Các mục sau phân tích chi tiết từng yêu cầu.

1.1.3 Mục tiêu và phạm vi

Mục tiêu của nhóm gồm:

1. Thiết kế kiến trúc dữ liệu end-to-end bao phủ tám yêu cầu nghiệp vụ, bám sát kiến trúc tham chiếu; mọi điều chỉnh đều có lý do và được ghi lại.
2. Hiện thực pipeline chạy trên máy cá nhân bằng Docker Compose: CDC thật qua Debezium, lakehouse Delta Lake với star schema, data mart theo từng BR và các đường xử lý thời gian thực.
3. Bảo đảm các tính chất cốt lõi: xử lý theo event time, chịu được late data và dữ liệu trùng lặp, rerun idempotent, pseudonymisation và k-anonymity, kiểm tra chất lượng dữ liệu.
4. Cung cấp API và giao diện web cho từng nhóm người dùng để kết quả pipeline được sử dụng trong thực tế.
5. Đánh giá tính đúng đắn và hiệu năng bằng kiểm thử tự động cùng các phép đo có thể tái lập.

Về phạm vi, báo cáo sử dụng **dữ liệu tổng hợp** (synthetic); hệ thống chạy trên một máy nên không nhằm chứng minh khả năng scale-out hay high availability. Các số đo hiệu năng là quan sát cục bộ, không phải cam kết về năng lực hệ thống.

1.1.4 Đóng góp chính

- **Lakehouse Delta Lake** ba lớp với 51 bảng và 44 model (43 model tại thời điểm đo thời gian build ở Chương 5, trước khi bổ sung model lịch sử chuyển trạng thái cảnh báo của BR5). Lớp gold là data warehouse Kimball gồm 9 dimension và 11 fact table, dùng surrogate key tất định. Bên cạnh đó là các mart cho BR4, BR6, BR7, BR8 và offline feature store cho BR2; Hive Metastore làm catalog, Trino phục vụ truy vấn có phân quyền theo vai trò.
- **Đường chạy thật cho BR1–BR3**: dịch vụ thông báo và chấm bài trên Kafka, Redis, PostgreSQL; feature store hai nhánh cho gợi ý.
- Đường **CDC thật** từ PostgreSQL WAL qua Debezium, Kafka và Spark Structured Streaming đến mart, trạng thái rủi ro và cảnh báo cho BR4–BR5; đường này chịu được replay, cập nhật, xoá và khởi động lại từ checkpoint.
- Đồng bộ nội dung **BR8 dựa trên CDC**: vô hiệu hoá Redis cache, chunk tài liệu, nhúng vector vào OpenSearch và tìm kiếm lai, idempotent theo phiên bản.

- **Overlay thời gian thực** cho heatmap video (BR7) và điểm tương tác (BR8) trên Redis, được ghép với mart theo ngày.
- Mười sáu **ADR**, kiến trúc DDD/hexagonal có test bảo vệ ranh giới cùng bộ kiểm thử nhiều tầng và bằng chứng đo đạc.
- **Giao diện web** cho năm workspace, được thiết kế từ mockup và nối với API thật.

1.1.5 Phương pháp thực hiện

Nhóm làm việc theo các vòng lặp ngắn: phân tích yêu cầu, ghi ADR, hiện thực, kiểm chứng bằng test và số đo. Lõi nghiệp vụ (policy và use case) được viết trước và kiểm thử độc lập với engine; sau đó nhóm bổ sung dần adapter cho Kafka, Spark, Delta, Redis, PostgreSQL và OpenSearch. Mỗi nhóm BR có tài liệu kế hoạch và tài liệu hiện thực riêng, ghi rõ phần đã hiện thực, phần còn hạn chế và các lỗi phát hiện trong quá trình kiểm chứng.

1.1.6 Cấu trúc báo cáo

Báo cáo gồm sáu chương:

- Chương 1: người dùng, yêu cầu nghiệp vụ, nguồn dữ liệu và yêu cầu phi chức năng.
- Chương 2: kiến trúc, lựa chọn công nghệ và các quyết định thiết kế.
- Chương 3: lakehouse, mô hình medallion, data warehouse và data mart.
- Chương 4: hiện thực BR1–BR8 và giao diện ứng dụng.
- Chương 5: kiểm thử, đánh giá hiệu năng và benchmark Debezium–polling.
- Chương 6: hạn chế, hướng phát triển và kết luận.

1.2 Phân tích yêu cầu và nguồn dữ liệu

1.2.1 Các bên liên quan và yêu cầu nghiệp vụ

Nhóm xác định bốn nhóm người dùng và tám yêu cầu nghiệp vụ. Bảng 1.1 tóm tắt từng yêu cầu cùng nhu cầu kỹ thuật tương ứng theo cách đọc của nhóm.

Bảng 1.1. Yêu cầu nghiệp vụ và nhu cầu data engineering

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu data engineering
BR1	Người học	Nhận thông báo khi có bài tập mới, deadline sắp đến hoặc nội dung được cập nhật	Pipeline thông báo hướng sự kiện: nhận sự kiện LMS qua message broker, lọc bằng rule engine, gửi trong thời gian thực
BR2	Người học	Nhận gợi ý khoá học và bước học tiếp theo	Pipeline feature hai nhánh: batch feature (Spark) và session feature (streaming) qua feature store
BR3	Người học	Nhận phản hồi chi tiết khi nộp bài lập trình hoặc SQL	Tiếp nhận bài nộp tách rời qua queue; chuẩn hoá kết quả test (stderr, diff test case, thời gian chạy)
BR4	Giảng viên	Phát hiện câu hỏi hoặc test case có tỷ lệ trượt bất thường	Aggregation mart: tỷ lệ đạt theo từng test case và câu hỏi nhằm phát hiện đề lỗi logic hoặc quá khó
BR5	Cố vấn học tập	Nhận cảnh báo sớm người học có nguy cơ bỏ dở	CDC và sliding window: đọc thay đổi từ OLTP, tính điểm rủi ro, phát sinh cảnh báo
BR6	Quản trị viên (B2B)	Theo dõi tiến độ kỹ năng và tỷ lệ hoàn thành theo tổ chức	Pipeline medallion: chuẩn hoá telemetry có schema versioning, pseudonymisation, báo cáo tổ chức với ngưỡng nhóm tối thiểu
BR7	Giảng viên	Xem heatmap tương tác video bài giảng	Tổng hợp chi tiết: mật độ play/pause/seek theo từng giây của video
BR8	Giảng viên	Đồng bộ trạng thái, vô hiệu hoá cache và làm mới search index khi nội dung thay đổi	Pipeline xu hướng tương tác: tổng hợp hoạt động theo window, tính engagement score, phát hiện xu hướng giảm

Nhận xét về BR8

Yêu cầu nghiệp vụ của BR8 (đồng bộ cache và search index khi nội dung thay đổi) và nhu cầu kỹ thuật đi kèm (pipeline xu hướng tương tác) là hai nhu cầu khác nhau. Nhóm hiện thực **cả hai**: đồng bộ nội dung dựa trên CDC (Mục 4.1.8.1) và mart xu hướng tương tác (Mục 4.1.8.2).

1.2.2 Nguồn dữ liệu

Bảng 1.2 mô tả bốn nguồn dữ liệu thô, đặc tính của chúng và cách nền tảng tiếp nhận. Mỗi nguồn có thách thức riêng, qua đó quyết định thiết kế của tầng ingestion.

Bảng 1.2. Bốn nguồn dữ liệu và cách tiếp nhận

Nguồn	Đặc tính	Thách thức	Tiếp nhận
Telemetry (web, Android, iOS)	JSON bán cấu trúc kiểu xAPI [2], lưu lượng lớn	Đến muộn, out-of-order (thiết bị offline), gửi lại, nhiều phiên bản schema, chứa PII, clock skew trên thiết bị	Topic `telemetry.events.raw.v1`, key theo người học; DLQ `telemetry.events.dlq.v1`
CDC từ PostgreSQL	Thay đổi cấp dòng từ WAL: tiến độ, bài nộp, điểm	Giữ thứ tự theo key, không ảnh hưởng truy vấn production, xử lý cả update và delete	Debezium (plugin `pgoutput`, replication slot) vào 11 topic `cdc.lms.public.*`
Kho học liệu (object storage)	Transcript, slide, PDF; văn bản dài	Chunk theo ngữ nghĩa, quản lý phiên bản chặt chẽ	Đối tượng bất biến theo phiên bản trong MinIO; thay đổi đi qua CDC của bảng `course_content`
Log sandbox	stdout, stderr, thời gian chạy, bộ nhớ, trạng thái assertion	Chuẩn hoá output, tách lỗi hạ tầng khỏi lỗi của người học	Topic `grading.submission_received.v1`, `grading.sandbox_result.raw.v1`, `grading.submission_graded.v1`

Catalog Kafka gồm 26 topic, được khai báo dưới dạng mã. Ngoài các topic trên còn có các topic sự kiện miền: `lms.*` (bài tập mới, sắp đến hạn, khoá học cập nhật), `risk.*` (điểm rủi ro, cảnh báo), `notification.*` và `content.*`. Runtime BR4/BR5 tự tạo thêm các topic riêng khi khởi tạo, nằm ngoài catalog này: chín topic CDC `br45.cdc.br45.<table>`, `br45.activity.v1`, `br45.clean.v2`, `br45.analytics.v2` và `risk.alert_transition.v1`.

1.2.3 Yêu cầu phi chức năng

Từ đặc tính nguồn dữ liệu và nhu cầu người dùng, nhóm rút ra các yêu cầu phi chức năng sau, dùng làm tiêu chí đánh giá các quyết định kiến trúc ở Chương 2.

Latency phù hợp từng BR
Thông báo, phản hồi chấm bài và cảnh báo rủi ro cần latency tính bằng giây. Báo cáo tổ chức và heatmap chấp nhận cập nhật theo ngày, kèm overlay cho hôm nay. Tra cứu feature cho gợi ý cần dưới 10 ms.

Exactly-once, idempotent
Việc gửi lại từ Kafka, rerun job và backfill không được làm sai số liệu; mọi lệnh batch phải idempotent theo ngày chạy.

Xử lý theo event time
Window được tính theo thời điểm sự kiện và có watermark; late data vẫn được lưu và gắn cờ thay vì bị bỏ qua.

Quyền riêng tư
Định danh thật chỉ tồn tại ở lớp thô và bị hạn chế truy cập; lớp phân tích chỉ dùng pseudonym; báo cáo tổ chức không công bố số liệu của nhóm dưới $k = 5$ người.

Schema evolution
Payload từ client có nhiều phiên bản; hệ thống chuyển đổi về phiên bản hiện hành hoặc từ chối kèm lý do.

Tái lập và kiểm toán
Có thể rebuild dữ liệu từ lớp thô, xem trạng thái cũ (time travel) và truy vết từng lần build.

Chất lượng dữ liệu
Quality gate được kiểm tra sau mỗi lớp; lỗi nghiêm trọng dừng pipeline.

Chạy được trên máy cá nhân
Toàn hệ thống chạy bằng Docker Compose, chỉ dùng CPU và không phụ thuộc dịch vụ trả phí.

1.2.4 Ánh xạ yêu cầu vào pipeline

Bảng 1.3 ánh xạ mỗi BR vào bounded context, runtime xử lý và nguồn phục vụ; cột cuối ghi mức độ hoàn thiện thực tế, được giải thích ở Chương 6.

Bảng 1.3. Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime

BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
1	notification	Dịch vụ Kafka + rule engine; fan-out theo PostgreSQL	Redis (dedup, inbox); SSE	Chạy thật
2	recommendation	Spark batch + session streaming; xếp hạng bằng rule	Delta, Redis, API	Chạy thật
3	grading	Kafka + grading-ingest/worker/normalizer	Redis; PostgreSQL `grading.*`	Chạy thật
4	analytics + br45	CDC, bảng Delta, job Spark SQL bất đồng bộ	PostgreSQL `analytics`; lịch sử ở `mart.assessment_item_stats`	Chạy thật
5	risk + br45	CDC, Spark Structured Streaming (window 7 ngày)	PostgreSQL; lịch sử ở `gold.fct_risk_score_daily`	Chạy thật
6	telemetry + analytics	Medallion Spark	Delta, Trino (`mart.org_progress`)	Chạy thật
7	analytics	Spark batch + streaming overlay	Delta, Trino, Redis	Chạy thật
8	content + analytics	CDC, dịch vụ content-sync; Spark batch + overlay	Redis, OpenSearch, Delta, Trino	Chạy thật

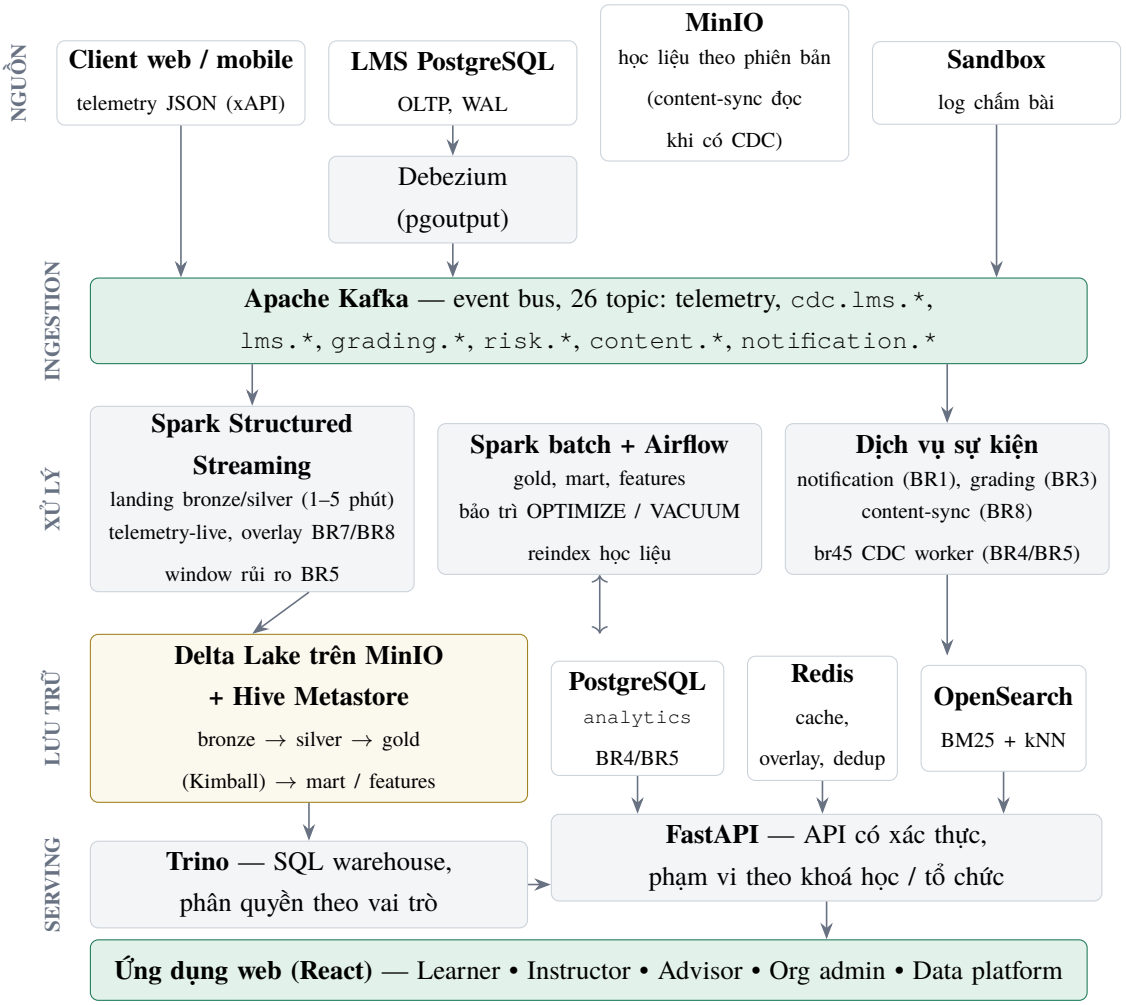
Chương 2

Kiến trúc và lựa chọn công nghệ

2.1 Kiến trúc tổng thể

2.1.1 Sáu tầng của kiến trúc

Hình 2.1 mô tả kiến trúc end-to-end gồm sáu tầng. Tầng nguồn gồm bốn nguồn dữ liệu; tầng ingestion dùng Kafka làm event bus duy nhất, với Debezium bắt thay đổi từ WAL của PostgreSQL; tầng xử lý dùng Spark cho cả streaming và batch, bên cạnh các dịch vụ tiêu thụ sự kiện; tầng lưu trữ gồm Delta Lake, PostgreSQL cho mart thời gian thực BR4–BR5, Redis và OpenSearch; tầng serving gồm Trino và API FastAPI; trên cùng là ứng dụng web cho năm nhóm người dùng.



Hình 2.1. Kiến trúc tổng thể của nền tảng Project

Ba nguyên tắc xuyên suốt kiến trúc:

- **Kafka là ranh giới giữa các bounded context.** Các context chỉ giao tiếp qua topic Kafka và bảng lakehouse, không dùng chung cơ sở dữ liệu; nhờ đó mỗi context có thể deploy và thay đổi độc lập.
- **Spark là writer duy nhất của lakehouse** (ADR 0011). Các job thời gian thực chỉ ghi ra Kafka và Redis. Nguyên tắc này tránh xung đột commit khi nhiều engine cùng ghi một bảng Delta, đồng thời giữ logic idempotent ở một nơi.
- **Lakehouse là source of truth cho phân tích.** Redis và các mart phục vụ đều có thể rebuild từ lakehouse hoặc từ dữ liệu thô (ADR 0006).

2.1.2 Lựa chọn công nghệ

Bảng 2.1 liệt kê công nghệ theo vai trò. Các thành phần chính gồm Kafka [3], Debezium [4], Spark [5] với Structured Streaming [6], Delta Lake [7], Trino [8], Redis [9] và OpenSearch [10].

Nhóm giữ các lựa chọn công nghệ ban đầu nếu chúng chạy được trên máy cá nhân; khi phải thay thế, nhóm chọn thành phần cùng vai trò và ghi lại quyết định bằng ADR.

Bảng 2.1. Công nghệ theo vai trò

Vai trò	Công nghệ	Lý do chọn
Ingestion, event bus	Apache Kafka	Buffer cho telemetry tốc độ cao, giữ thứ tự theo key, replay được; đồng thời là event bus cho các sự kiện LMS (ADR 0003)
CDC	Debezium (pgoutput)	Đọc replication slot nên không tạo truy vấn lên production; lưu LSN để sắp thứ tự
Stream processing	Spark Structured Streaming	Watermark theo event time, checkpoint, sink giao dịch Delta; dùng chung engine với batch (ADR 0009, 0014)
Batch processing	Apache Spark 3.5	Hệ sinh thái Delta, song song hoá model SQL và bước chunk học liệu (ADR 0004)
Lakehouse	Delta Lake 3.3 trên MinIO	ACID MERGE, time travel, Change Data Feed (ADR 0005)
Catalog	Hive Metastore 3.1.3	Catalog chung cho Spark và Trino; bản 3.1 tương thích Spark 3.5 (ADR 0010)
SQL warehouse	Trino	Truy vấn trực tiếp bảng Delta, ANSI SQL, phân quyền theo schema, không cần bản sao thứ hai (ADR 0010)
Serving latency thấp	Redis	Tra cứu theo key dưới mili-giây, TTL; dùng cho cache, overlay thời gian thực và dedup thông báo (ADR 0006)
Mart thời gian thực BR4/BR5	PostgreSQL 16	Cập nhật có tính transaction; sau mỗi thay đổi của nguồn, kết quả được ghi bất đồng bộ trong một transaction (ADR 0009)
Tìm kiếm	OpenSearch 2.17	BM25 kết hợp vector HNSW; xếp hạng lai bằng reciprocal rank fusion (ADR 0013)
Điều phối	Apache Airflow	DAG theo lịch và theo dataset; mọi task idempotent theo ngày chạy
API, giao diện	FastAPI, React 18 + TypeScript	Một API chung cho mọi BR; giao diện không dùng thư viện UI hay chart bên ngoài

2.1.3 Kiến trúc phần mềm: DDD và hexagonal

Mã nguồn nằm trong một gói Python duy nhất (monorepo, ADR 0002), chia thành bảy bounded context theo Domain-Driven Design [11]: `telemetry`, `notification`, `recommendation`, `grading`, `risk`, `analytics` và `content`. Mỗi context gồm bốn lớp theo kiến trúc hexagonal [12] (Bảng 2.2); một architecture test kiểm tra quy tắc phụ thuộc giữa các lớp.

Bảng 2.2. Các lớp trong một bounded context

Lớp	Chứa gì	Được phụ thuộc vào
Domain layer	Entity, value object, aggregate, domain event, policy (ví dụ `EngagementPolicy`, `MinCohortPolicy`)	Chỉ `shared_kernel.domain`
Application layer	Use case (mỗi lớp một hàm `execute`), các port `Protocol`	domain
Infrastructure layer	Adapter: Kafka, Redis, Delta, S3, Trino, OpenSearch, YAML	Mọi thứ
Entrypoint layer	Điểm ghép nối cho job Spark, dịch vụ, API	Mọi thứ

Lợi ích cốt lõi của cách tổ chức này là **một logic, nhiều engine**. Use case `NormalizeEvents` (kiểm tra hợp lệ, migrate schema, pseudonymise, gắn cờ late data) chạy trong job streaming, job silver của lakehouse và dịch vụ xử lý sự kiện, cho cùng kết quả về pseudonym và quyết định đưa vào DLQ. Các ngưỡng nghiệp vụ trong SQL mart lấy từ value object của domain và có unit test bảo đảm luôn đồng bộ.

2.1.4 Ngữ nghĩa stream processing

- **Event time được dùng ở mọi nơi** theo mô hình Dataflow [13]. Watermark cho phép late data tối đa 5 phút với telemetry và 10 giây với CDC. Riêng window rủi ro BR5 dùng watermark 21 ngày vì dữ liệu học tập có thể được đồng bộ muộn.
- **Late data không bị mất.** Sự kiện đến muộn hơn watermark vẫn được lưu và gắn cờ. Các model gold tự mở rộng window tính toán về ngày của sự kiện muộn nhất (cơ chế `late_data`, Mục 3.1.8).
- **Exactly-once.** Landing dùng checkpoint của Structured Streaming kết hợp sink giao dịch Delta. Lớp silver dùng MERGE idempotent theo business key. Overlay trên Redis ghi giá trị tuyệt đối thay vì cộng dồn, nên replay không đếm hai lần.
- **Thứ tự CDC.** Thay đổi được sắp theo `source.lsn` cho từng key; thay đổi có LSN cũ hơn

không thể ghi đè bản ghi mới hơn.

- **Clock skew trên thiết bị.** Sự kiện có thời gian đi trước thời điểm tiếp nhận hơn 2 phút được ghim về thời điểm tiếp nhận (`ClockSkewPolicy`). Nếu không, một thiết bị có đồng hồ chạy nhanh sẽ đẩy watermark lên và làm rơi các sự kiện đúng hạn của những người dùng khác.

2.1.5 Các quyết định kiến trúc

Mỗi quyết định quan trọng được ghi thành một ADR [14] trong bộ ADR của dự án. ADR đã chấp nhận thì không sửa; khi cần thay đổi, nhóm viết ADR mới thay thế ADR cũ. Bảng 2.3 tóm tắt mười sáu ADR.

Bảng 2.3. Tóm tắt các quyết định kiến trúc (ADR)

ADR	Quyết định	Lý do và hệ quả chính
0001	Ghi quyết định bằng ADR	Tránh tranh luận lại; lưu giữ lý do thay vì chỉ lưu kết quả
0002	Monorepo Python, DDD + hexagonal	Use case độc lập với engine; đổi lại là một chi phí nhỏ cho các lớp gián tiếp
0003	Kafka + Debezium cho ingestion và CDC	Giữ thứ tự theo entity; phải giám sát replication slot
0004	Flink cho streaming, Spark cho batch	Lựa chọn ban đầu của nhóm; sau đó được điều chỉnh bởi ADR 0009 và 0014
0005	Lakehouse Delta Lake theo medallion	ACID upsert, time travel, ranh giới PII; cần job bảo trì để chống small file
0006	Redis cho serving layer	Dưới mili-giây, TTL; Redis không phải source of truth
0007	Pseudonymisation ở ranh giới silver	HMAC-SHA256 với key có phiên bản; báo cáo tổ chức áp dụng $k = 5$
0008	Runtime ban đầu chạy trong tiến trình	Trình diễn end-to-end trên laptop bằng adapter trong bộ nhớ; BR1–BR3 sau đó chuyển sang runtime thật (ADR 0016)
0009	PostgreSQL mart + Spark Structured Streaming cho BR4/BR5	Một driver Spark trên đường sản phẩm, chạy cả window BR5 lẫn job tổng hợp BR4; job Flink tương đương chỉ dùng để so sánh engine; latency từ CDC đến cảnh báo khoảng 10 giây (không đo lại)
0010	Trino + Hive Metastore làm SQL warehouse	Catalog và SQL warehouse đặt trên chính các bảng Delta, không có bản sao thứ hai

ADR	Quyết định	Lý do và hệ quả chính
0011	Spark là writer duy nhất của Delta	Connector Flink cho Delta đã lỗi thời; đồng thời tránh xung đột commit
0012	Gold là star schema Kimball, key tất định	`xxhash64` của natural key; rebuild tái tạo đúng key; dùng inferred member cho dimension đến muộn
0013	Đồng bộ nội dung BR8 bằng CDC vào OpenSearch	Một đường đồng bộ cho mọi writer; idempotent theo phiên bản đã index
0014	Spark Structured Streaming cho telemetry nhanh và overlay BR7/BR8	Chưa có job Flink cho telemetry và overlay; latency đo được khoảng 6 giây là đủ cho overlay
0015	Session feature BR2 trên Spark Structured Streaming + Redis	Spark 3.5 không hỗ trợ `update` cho `session_window`; session state nằm trong Redis, `SET NX` chống đếm lại
0016	Runtime thật cho BR1–BR3	Kafka, Redis, PostgreSQL thật; kết quả đến UI qua SSE; projector BR3 đồng bộ `public.*`

2.1.6 Điều chỉnh so với kiến trúc tham chiếu

Nhóm giữ nguyên vai trò của kiến trúc tham chiếu nhưng điều chỉnh ba điểm:

1. **Dùng Spark Structured Streaming thay Flink cho BR5 và overlay BR7–BR8** (ADR 0009, 0014). Kiến trúc tham chiếu dùng Flink [15] cho stateful stream processing; khi bước vào hiện thực, job Flink mới chỉ là skeleton trong khi Spark Structured Streaming đã chạy cho landing lakehouse. Dùng một engine giúp giảm hạ tầng vận hành và cho phép tái sử dụng code chuẩn hoá silver. Đổi lại, latency khoảng 6 giây cho overlay và khoảng 10 giây cho cảnh báo BR5 (số của BR5 không đo lại) cao hơn mức dưới một giây của Flink, nhưng phù hợp với cảnh báo dành cho cố vấn và overlay trên mart theo ngày. Về sau, nhóm viết thêm một job Flink tính cùng các window rủi ro (`infra/br45/flink/RiskWindows.java`) để so sánh engine cho BR5; job này chạy trong một stack benchmark riêng (`docker-compose.br45-benchmark.yml`), chỉ ghi vào bảng benchmark và không sinh cảnh báo, còn đường sản phẩm vẫn là Spark Structured Streaming. Cụm Flink vẫn có trong Compose chính, chưa chạy job nào, để sẵn sàng chuyển đổi khi cần.
2. **Bổ sung Trino + Hive Metastore làm catalog và SQL warehouse** (ADR 0010). Delta Lake chỉ là định dạng bảng, nên data warehouse cần thêm một catalog và một engine truy vấn. Hive Metastore và Trino đảm nhận hai vai trò này trên chính các bảng Delta, không

tạo bản sao thứ hai và vẫn chạy được cục bộ.

3. **BR2 dùng feature store hai nhánh.** Yêu cầu BR2 mô tả pipeline feature hai nhánh qua feature store; nhóm hiện thực cả hai: batch ghi bảng `features.learner_longterm`, streaming giữ session state trong Redis. Gợi ý dùng feature làm ngữ cảnh và xếp hạng bằng rule nghiệp vụ (Mục 4.1.2).

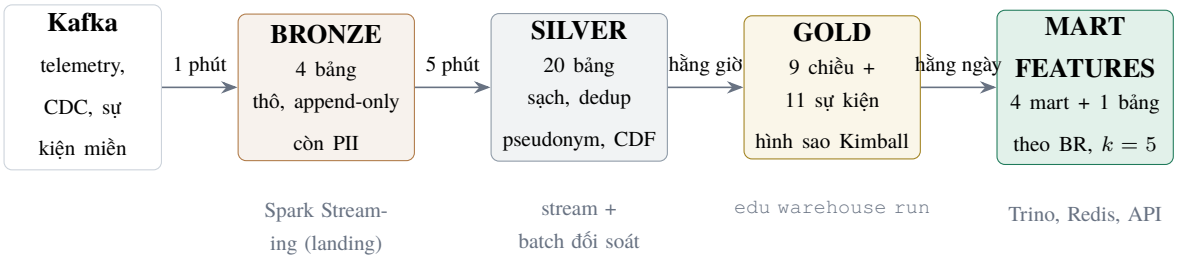
Chương 3

Lakehouse và data warehouse

3.1 Tổng quan lakehouse

Chương này trình bày lakehouse [16] Delta Lake, được tổ chức theo mô hình medallion [17] với data warehouse chính là lớp gold. Nền tảng không có cơ sở dữ liệu phân tích thứ hai: Spark ghi bảng Delta, Hive Metastore làm catalog, Trino phục vụ SQL và Redis xử lý các tra cứu mili-giây. Toàn bộ DDL, model, quality gate và cấu hình đều được quản lý dưới dạng mã.

3.1.1 Kiến trúc medallion



Hình 3.1. Các lớp của lakehouse và tần suất cập nhật

Bảng 3.1. Data contract của từng lớp

Schema	Số bảng	Writer	Contract
`bronze`	4	Stream landing	Mỗi bản ghi Kafka tương ứng một dòng, giữ nguyên như khi nhận. Chỉ append, còn PII, chỉ vai trò `pii_steward` được đọc. Lưu 30 ngày (telemetry) hoặc 90 ngày (CDC, sự kiện). Replay bronze có thể rebuild mọi lớp phía sau.
`silver`	19	Stream silver + batch đối soát	Có kiểu dữ liệu, đã kiểm tra hợp lệ, dedup, đúng schema hiện hành, đã pseudonymise bằng HMAC; định danh trực tiếp bị loại bỏ. CDC áp dụng theo thứ tự LSN, xoá mềm. Change Data Feed được bật.
`gold`	9 + 11	`edu warehouse run`	Star schema Kimball (ADR 0012): dimension dùng chung, surrogate key tất định, fact table có grain rõ ràng.
`mart`	4	`--layer mart`	Mỗi BR một bảng, thiết kế theo nhu cầu người đọc; ngưỡng lấy từ policy của domain. `mart.org_progress` là bảng duy nhất mà quản trị viên B2B được đọc.
`features`	1	`--layer features`	Offline feature store cho BR2, mỗi `as_of_date` là một partition để huấn luyện đúng thời điểm (point-in-time).
`ops`	2	Runner	`pipeline_runs` (mỗi lần build: window, số dòng, phiên bản Delta) và `dq_results` (kết quả kiểm tra chất lượng).

3.1.2 Chuẩn hoá telemetry và schema evolution

Telemetry đi qua use case `NormalizeEvents`, dùng chung cho job silver và job thời gian thực:

1. **Kiểm tra hợp lệ** theo schema contract: thiếu trường bắt buộc, verb nằm ngoài từ vựng hoặc payload hỏng đều bị đưa vào DLQ kèm lý do.
2. **Migrate schema**: `SchemaMigrator` nâng phiên bản 1.0 lên 1.1; phiên bản không nhận diện được bị từ chối.
3. **Pseudonymisation**: thay định danh người học bằng pseudonym; loại bỏ email, họ tên, IP

và mã thiết bị.

4. **Xử lý thời gian**: tính độ trễ so với thời điểm tiếp nhận, gắn cờ `late` và ghim các sự kiện có đồng hồ chạy trước.
5. **Dedup**: MERGE theo `event_id` (chỉ insert) nhằm loại các lần gửi lại từ thiết bị offline.

3.1.3 Pseudonymisation và phân quyền

Định danh thật chỉ tồn tại ở bronze (ADR 0007); từ silver trở đi, người học được biểu diễn bằng pseudonym `k<version>_<24 hex>` sinh từ HMAC-SHA256 với key có phiên bản. Hàm SQL `pseudonymize()` dùng cùng thuật toán và key với `HmacPseudonymizer` của context telemetry, nhờ đó telemetry, CDC và sự kiện miền vẫn join được với nhau. Quality gate `silver_no_raw_identifiers` chặn định danh thô lọt qua bronze.

Trino áp dụng phân quyền chỉ đọc theo schema (Bảng 3.2). Quản trị viên B2B chỉ thấy đúng một bảng đã áp dụng k-anonymity.

Bảng 3.2. Phân quyền truy cập data warehouse qua Trino

Người dùng	bronze	silver	gold	mart	features / ops
`pii_steward`	✓	✓	✓	✓	✓
`analyst`	–	✓	✓	✓	✓
`app` (API, dashboard)	–	–	✓	✓	–
`b2b_admin`	–	–	–	chỉ `org_progress`	–

3.1.4 Star schema Kimball ở lớp gold

Tám yêu cầu cùng đọc những sự kiện chồng lấn (học tập, bài nộp, ghi danh, rủi ro) theo các thực thể chung (người học, khoá học, nội dung, bài tập, tổ chức). Nếu thiếu dimension dùng chung, mỗi mart sẽ tự suy ra thực thể theo cách riêng và dần lệch nhau; vì vậy lớp gold được mô hình hoá thành star schema Kimball [18] (ADR 0012). Bảng 3.3 là bus matrix cho biết mỗi fact table sử dụng những dimension nào.

Bảng 3.3. Bus matrix của lớp gold (D: vai trò ngày, ●: khoá dimension)

Fact table (grain)	*date*	*learner*	*course*	*content*	*activity*	*lesson*	*assignment*	*test case*	*org*	BR
`fct_learning_event` (một sự kiện)	D	●	●	●	●					6,7,8
`fct_learner_daily_activity` (người học × khoá × ngày)	D	●	●							2,8
`fct_video_second_activity` (phiên bản × giây × ngày)	D			●						7
`fct_submission` (một lần nộp)	D	●	●				●			3,4
`fct_test_case_result` (lần nộp × test)	D	●					●	●		4
`fct_lesson_progress` (người học × bài)		●	●			●				6
`fct_enrollment` (một lần ghi danh)	D×5	●	●						●	5,6
`fct_risk_score_daily` (người học × khoá × ngày)	D	●	●							5
`fct_dropout_alert` (một cảnh báo)	D	●	●							5
`fct_notification` (một kết quả gửi)	D	●								1
`fct_content_sync` (một phiên bản nội dung)			●	●						8

Dimension và surrogate key

- **SCD type 2** cho `dim_learner` và `dim_content`. Với `dim_learner`, báo cáo B2B phải quy hoạt động về tổ chức tại đúng thời điểm xảy ra. Với `dim_content`, mỗi phiên bản video có một dòng riêng vì video quay lại có độ dài khác nhau, khiến giây thứ n của hai phiên bản không còn ứng với cùng một đoạn video (BR7). Các dimension còn lại dùng SCD type 1.
- **Surrogate key tất định**: `xxhash64(natural key)` với SCD1 và `xxhash64(natural key, valid_from)` với SCD2. Rebuild một dimension sẽ tạo lại đúng các key cũ, nên

dimension và fact table có thể build độc lập mà không cần dịch vụ sinh số tuần tự.

- **Inferred member** cho dimension đến muộn: telemetry của một người học khi dòng CDC chưa đến sẽ tạo một dòng `is_inferred = true` mang đúng key mà dòng thật sẽ có. Khi dòng thật đến, nó ghi đè tại chỗ nên không fact table nào phải đổi key. Điều này đã được kiểm chứng: người học `1-131` có sự kiện trước dòng CDC hai ngày vẫn giữ nguyên key qua toàn bộ quá trình chuyển đổi.
- Key `-1` biểu diễn "không áp dụng", ví dụ nội dung của một sự kiện đăng nhập.

Một số quy tắc dùng chung: bài học được coi là hoàn thành khi CDC báo `percent_done = 100` *hoặc* telemetry có sự kiện `completed`, tuỳ sự kiện nào đến trước; vị trí video vượt quá độ dài của phiên bản bị kẹp về giây cuối; lần chạy sandbox mới nhất thay thế các lần trước.

3.1.5 Data mart theo yêu cầu nghiệp vụ

Mỗi mart hiện thực một use case phân tích bằng SQL tổng hợp. Các ngưỡng được truyền từ value object của domain (`policy_params`), ghi trên từng dòng và có unit test đối chiếu với domain.

Bảng 3.4. Các data mart và feature store

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`mart.assessment_item_stats`	BR4, giảng viên	Tỷ lệ đạt theo (bài tập, test case) trong 28 ngày; bất thường khi có ≥ 30 lượt, tỷ lệ đạt $\leq 0{,}35$ và thấp hơn median của bài tập ít nhất $0{,}30$	`AnomalyThreshold`
`mart.org_progress`	BR6, quản trị B2B	Theo tổ chức × khoá học: quy mô nhóm, tỷ lệ hoàn thành, mức thành thạo trung bình; **nhóm dưới k người chỉ công bố quy mô** [19]	`MinCohortPolicy(k=5)`
`mart.video_heatmap`	BR7, giảng viên	Lưới dày theo từng giây cho mỗi phiên bản video trong 28 ngày; số người học phân biệt chính xác; cường độ 0–1	–
`mart.engagement_trend`	BR8, giảng viên	4 window tuần cho mỗi người học × khoá; "giảm" khi điểm giảm $\geq 30\%$ trong mỗi window của hai tuần gần nhất	`EngagementWeights`, tỷ lệ 0,30

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`features.learner_longterm`	BR2, gợi ý	`ComputeBatchFeatures` trong 30 ngày, theo `as_of_date`	registry feature v1

BR1, BR3 và BR5 được phục vụ thời gian thực qua dịch vụ thông báo, dịch vụ chấm bài và runtime BR4–BR5 (ADR 0009). Song song với đó, data warehouse lưu lịch sử phân tích: `fct_risk_score_daily` và `fct_dropout_alert` có nhãn `dropped_within_30d` để đánh giá độ chính xác của cảnh báo; `fct_notification` và `fct_submission` phục vụ BR1 và BR3.

3.1.6 Thiết kế vật lý

- **Tính năng Delta**: ACID MERGE, `INSERT ... REPLACE WHERE` cho window ngày, ghi đè partition động, Change Data Feed ở silver, time travel (`FOR VERSION AS OF`) trong phạm vi VACUUM 7 ngày.
- **Partition** chỉ theo cột ngày mà truy vấn có lọc (ngày sự kiện, ngày hoạt động, ngày nộp, `as_of_date`); bảng nhỏ không partition; bronze partition theo `ingest_date`.
- **Clustering**: thuộc tính bảng `edu.zorder` được job bảo trì hằng ngày áp dụng bằng `OPTIMIZE ... ZORDER BY`. Cùng job này gộp các small file do writer streaming sinh ra.
- **Policy là thuộc tính bảng**: `edu.pii`, `edu.owner`, `edu.retention_days`, `edu.zorder` hiển thị cho mọi engine (`SHOW TBLPROPERTIES`, bảng `$properties` của Trino).
- **Schema evolution tường minh**: tắt `autoMerge`; mỗi thay đổi DDL đi kèm câu lệnh migration.

3.1.7 Quality gate

Sau mỗi lớp, runner chạy bộ truy vấn kiểm tra chất lượng và ghi kết quả vào `ops.dq_results`; quality gate mức `error` sẽ dừng lần chạy.

Bảng 3.5. Các quality gate

Gate	Mức	Điều gì được bảo vệ
`silver_event_id_unique`	error	Dedup các lần gửi lại từ mobile
`silver_no_raw_identifiers`	error	Ranh giới PII: chỉ còn pseudonym sau bronze
`bronze_dlq_rate`	warn	Hơn 5% telemetry bị từ chối
`gold_dim_learner_scd2`	error	Mỗi người học đúng một phiên bản hiện hành, không chồng lấn hiệu lực
`gold_orphan_keys`	error	Mọi key trong fact table đều tồn tại trong dimension
`gold_unresolved_learners`	warn	Sự kiện bị quy về thành viên "không rõ"
`mart_org_progress_k_anonymity`	error	Không công bố tỷ lệ cho nhóm dưới k người
`mart_rates_in_range`	error	Các tỷ lệ nằm trong $[0, 1]$
`mart_org_progress_unresolved_dims`	warn	Dòng báo cáo thiếu khoá học hoặc kỹ năng
`mart_video_heatmap_unversioned`	warn	Ô heatmap không gắn với phiên bản video

3.1.8 Điều phối, late data và rerun

Job streaming và các dịch vụ chạy liên tục; job batch do Airflow điều phối (Bảng 3.6). Mọi lệnh batch đều idempotent theo `--run-date`, nên retry và backfill an toàn; `--start-date` cho phép rebuild bất kỳ khoảng thời gian nào.

Bảng 3.6. Các DAG Airflow

DAG	Lịch	Công việc
`medallion_daily`	hằng giờ	đối soát silver, build gold
`analytics_marts_daily`	02:00 hằng ngày	làm mới gold, mart BR4 / BR6 / BR7 cho ngày đã đóng
`engagement_weekly`	thứ Hai 02:00	`mart.engagement_trend`
`features_daily`	khi gold cập nhật	`features.learner_longterm`
`content_embedding`	6 giờ một lần	chunk và nhúng học liệu
`lakehouse_maintenance`	04:00 hằng ngày	xoá theo retention, OPTIMIZE / Z-order, VACUUM

Bảng 3.7. Cơ chế xử lý late data, dữ liệu trùng lặp và out-of-order

Tình huống	Lớp	Cơ chế
Kafka gửi lại, job rerun	bronze	Checkpoint Structured Streaming + sink Delta (exactly-once)
Client gửi lại (mobile offline)	silver	MERGE theo `event_id`; quality gate `silver_event_id_unique`
CDC out-of-order hoặc bị replay	silver	Lấy thay đổi mới nhất theo `lsn`; chỉ MERGE khi `lsn` mới hơn; replay không tạo commit Delta
Delete ở OLTP	silver	Debezium ở chế độ `rewrite`: `is_deleted = true` (xoá mềm)
Late data (mobile đồng bộ sau vài ngày)	gold	Model theo window khai báo `late_data`; window tự lùi về ngày của sự kiện cũ nhất mà silver nhận được kể từ lần build trước (đã kiểm chứng: lần chạy 06–14/10 tự mở rộng về 04/10)
Sự kiện đến trước dimension	gold	Inferred member với key cuối cùng (ADR 0012)
Xoá theo retention ở bronze	silver	`skipChangeCommits` để thao tác xoá không làm dừng stream

Tình huống	Lớp	Cơ chế
Schema evolution phía client	silver	Chuỗi `SchemaMigrator` 1.0 → 1.1; không nhận diện được thì vào DLQ

Khi có sự cố, bronze đóng vai trò replay log: sửa SQL rồi chạy lại `edu warehouse run --start-date` cho lớp bị ảnh hưởng; khi cần trạng thái cũ, dùng time travel của Delta với phiên bản được lưu trong `ops.pipeline_runs`.

Chương 4

Hiện thực BR1–BR8 và ứng dụng

4.1 Thiết kế và hiện thực theo từng yêu cầu

Chương này trình bày tám yêu cầu nghiệp vụ. Với mỗi BR, báo cáo mô tả data flow, các rule nghiệp vụ quan trọng và mức độ hoàn thiện. BR1–BR8 đều chạy trên hạ tầng thật: Kafka, Redis, PostgreSQL, Spark, Delta, Trino và OpenSearch. Dịch vụ notification và grading dùng Kafka, Redis, PostgreSQL; BR2 dùng job batch `features-online` và job streaming `telemetry-live` trên Delta và Redis.

4.1.1 BR1 — Pipeline thông báo hướng sự kiện

Đường chạy thật. Dịch vụ `notification` là một tiến trình độc lập, tiêu thụ năm topic Kafka: `lms.assignment_created.v1`, `lms.deadline_approaching.v1`, `lms.course_updated.v1`, `grading.submission_graded.v1` và topic `risk.dropout_alert.v1` (mang event type `risk.dropout_risk_detected.v1`). Mỗi sự kiện đi qua rule engine, được đối chiếu với tập rule YAML rồi phân giải người nhận từ PostgreSQL (`public.learners`, `public.enrollments`) hoặc cố vấn của người học. Thông báo được ghi vào Redis và gửi qua in-app, push hoặc email. Kết quả được phát lên `notification.dispatched.v1` hoặc `notification.suppressed.v1`, đồng thời ghi vào lakehouse thành `gold.fct_notification`. API đọc inbox từ Redis; cầu Kafka–SSE đẩy sự kiện `notification` xuống trình duyệt. Vòng tiêu thụ có retry/backoff và DLQ, nên một sự kiện lỗi không làm dừng dịch vụ.

Rule engine. Rule được khai báo trong YAML và nạp khi dịch vụ khởi động (Hình 4.1). Điều kiện được viết bằng một ngôn ngữ biểu thức tối giản trên payload của sự kiện, không cho phép chạy mã tuỳ ý.

```
rules:
  - rule_id: deadline-24h
    event_type: lms.deadline_approaching.v1
    condition: "hours_to_deadline <= 24"
    channels: [push, email]
    priority: high
    template_id: deadline_soon
  - rule_id: dropout-risk-advisor
    event_type: risk.dropout_risk_detected.v1
    channels: [email]      # recipient = the learner's advisor
    priority: high
```

Hình 4.1. Trích tập rule thông báo

Chống làm phiền người dùng. Mỗi thông báo có dedup key trong Redis với TTL 24 giờ, nhờ đó cùng một sự kiện không được gửi hai lần. Rate limit dùng sliding window 5 thông báo/giờ cho mỗi người học; thông báo ưu tiên cao (ví dụ còn dưới 24 giờ đến hạn) được miễn giới hạn. Notification store idempotent theo định danh: hai use case route và dispatch đều gọi `save()` nhưng chỉ lần tạo đầu tiên thêm vào inbox và tăng counter, nên thông báo không bị nhân đôi. Các thông báo bị chặn được ghi kèm lý do để giao diện hiển thị.

Kênh gửi thật. In-app ghi vào Redis; email đi qua SMTP; push đi qua webhook HTTP, cả hai đều có retry ba lần. Trong lần kiểm chứng, email và webhook đều đến đích; provider được cấu hình qua biến môi trường.

Kết quả kiểm chứng. Đăng ký `l-002` vào `c-ml-301` ghi một dòng trong PostgreSQL; sự kiện deadline tạo đúng một thông báo; replay cùng sự kiện trả lý do `duplicate` và không tăng `notif:count`. Trong cùng đợt kiểm chứng, email và webhook đều gửi thành công; Playwright xác nhận điều khiển gửi thông báo trên giao diện.

4.1.2 BR2 — Gợi ý khoá học và bước học tiếp theo

Yêu cầu BR2 mô tả pipeline feature hai nhánh: feature dài hạn tính bằng batch trên Spark, feature phiên tính bằng streaming, cả hai đi qua feature store. Nhóm hiện thực đủ hai nhánh và nối chúng vào cùng một online store trên Redis.

Nhánh batch. Bảng `features.learner_longterm` gồm completion rate 30 ngày, average pass rate, số ngày hoạt động, nền tảng ưa dùng và mức thành thạo theo kỹ năng. Mỗi ngày được tính thành một partition, bảo đảm point-in-time correctness và không rò rỉ dữ liệu tương lai. Job `edu batch features-online` đọc partition mới nhất và ghi `edu:features:longterm:<pseudonym>` vào Redis với TTL 36 giờ. Lần materialise ngày 2026-11-10 ghi 40 key từ bảng

Delta.

Nhánh streaming. Job `edu stream session-features` (chạy trong `telemetry-live`) đọc `telemetry.learning_events.v1`, giữ session state trong Redis và tách session khi khoảng trống vượt quá 30 phút hoặc `session_id` thay đổi. Mỗi `event_id` được claim bằng `SET NX` trong 1 giờ, nên replay Kafka không đếm hai lần. Kết quả ghi `edu:features:session:<pseudonym>` với TTL 2 giờ và phát `features.session.v1` để lưu vết (ADR 0015). Micro-batch 30 giây giúp session đang mở hiển thị ngay mà không phải chờ watermark.

Serving. `LoadLearnerFeatures` kết hợp hai nhánh theo pseudonym. API trả `features.longterm` và `features.session`; giao diện hiển thị chip "Live session" và "% recent pass". `NextStepPolicy` dùng feature làm context: pass rate thấp làm giảm ưu tiên bước tiếp theo, session đang gặp khó thì ưu tiên bù kiến thức; khi hoà thì giữ nguyên lộ trình hiện tại. Khi context rỗng, rule giữ nguyên hành vi cũ.

Bộ xếp hạng dựa trên **rule nghiệp vụ**, được tính tại query time từ catalog khoá học và các khoá đã hoàn thành. Rule xếp các khoá chưa ghi danh theo *tier*, rồi theo relevance, difficulty và preference (Bảng 4.1); mỗi gợi ý kèm một câu giải thích để người học hiểu vì sao khoá đó được đề xuất.

Bảng 4.1. Các tier của rule gợi ý (BR2)

Tier	Trường hợp	Ví dụ lời giải thích	Điểm
0	Bước tiếp theo trong lộ trình	"Tiếp tục lộ trình data engineering: mức khó tiếp theo" hoặc "Bước liên quan trong lộ trình"	1,00
1	Mở rộng cùng mức	"Mở rộng ở mức của bạn: một khoá analytics mức dễ"	0,75
2	Bù khoá bị bỏ qua	"Bổ sung khoá mức dễ bạn đã bỏ qua"	0,50
3	Quá sức	"Quá sức: hoàn thành khoá mức trung bình trước" (khoá bị khoá)	0,00

Kết quả kiểm chứng. Năm sự kiện telemetry live của `l-002` tạo một session 5 sự kiện; thêm 2 sự kiện và chạy lại thì session có 7 sự kiện với cùng `started_at`, còn replay không đếm lại. API trả cả `longterm` lẫn `session` sau khi build warehouse và materialise 40 key. Sáu unit test phủ policy, store, phép kết hợp và xếp hạng theo feature; Playwright xác nhận trang người học và danh sách gợi ý hiển thị.

4.1.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết

Luồng tách rời, chạy thật. Giao diện gọi BFF `POST /api/submissions`; BFF chuyển tiếp sang `grading-ingest` ở cổng 8001. Dịch vụ ghi bài nộp ở trạng thái `QUEUED` vào Redis và

`grading.submissions`, phát `grading.submission_received.v1` và chỉ trả 202 sau khi broker xác nhận (`publisher.flush()`); nếu queue không nhận, API trả 503 thay vì nhận việc rồi làm mất. Worker nhận message, claim `SET NX`, chuyển trạng thái `RUNNING`, chạy sandbox và phát `grading.sandbox_result.raw.v1`. Normalizer áp `FeedbackPolicy`, ghi kết quả vào Redis và `grading.grades`, rồi phát `grading.submission_graded.v1`. Sự kiện này đi vào BR1, cầu SSE và projector đồng bộ `public.*`.

Lifecycle và idempotency. Trạng thái đi theo `queued` → `running` → `graded` hoặc `failed`. Worker claim theo `submission_id` nên message trùng không chạy sandbox lần hai; normalizer bỏ qua kết quả đã có theo `(submission_id, worker_run_id)`; PostgreSQL dùng UPSERT theo `submission_id`. Nhờ đó replay queue message hoặc raw result không tạo ra grade thứ hai.

Sandbox. Bài Python chạy trong một tiến trình con có time limit 3 giây. Bài SQL chạy trên SQLite in-memory ở chế độ read-only, có hàm ngắt khi quá hạn. Lỗi hạ tầng được tách khỏi lỗi trong code của người học: lỗi và timeout của code người học được tính là trượt, còn lỗi hạ tầng không tính vào pass rate.

Phản hồi có cấu trúc. `FeedbackPolicy` là một hàm thuần, không I/O, tạo phản hồi gồm câu tóm tắt, danh sách test trượt kèm giá trị mong đợi và thực tế (hoặc diff), 8 dòng cuối của stderr và gợi ý theo ngôn ngữ dựa trên bảng mẫu lỗi thường gặp. Kết quả còn gồm thời gian chạy, bộ nhớ và exit code. Mục 4.2 trình bày cách hiển thị phản hồi trong Code lab.

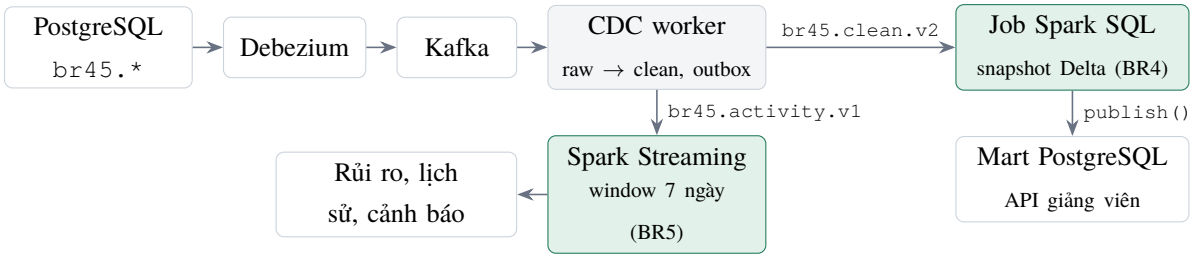
Sở hữu dữ liệu. BR3 ghi `grading.submissions` và `grading.grades`; projector `grading-projector` đồng bộ sang `public.submissions`/`public.grades` theo profile `br13-project` để LMS và CDC nhận được lịch sử.

Kết quả kiểm chứng. Nộp bài qua BFF trả 202 `queued`, sau đó chuyển `running` rồi `graded` với pass rate 0,8 và phản hồi "4/5 tests passed". Redis có đủ key submission, worker, result và code; PostgreSQL có dòng `graded` trong `grading.*`; inbox của `l-002` có thông báo từ `grading.submission_graded.v1`; SSE phát `submission.graded`. Integration test end-to-end đạt; Playwright 2/2 đạt. Projector đồng bộ `public.*` từ 9/9 lên 11/11 trong lần kiểm chứng.

4.1.4 BR4 — Phát hiện câu hỏi và test case bất thường

Đường CDC thật (ADR 0009). Dữ liệu vận hành nằm trong schema `br45` của PostgreSQL: khoá học, bài tập, câu hỏi và test case có version, bài nộp và kết quả chấm. Một connector Debezium riêng bắt WAL vào các topic `br45.cdc.br45.<table>`. CDC worker lưu envelope gốc vào `analytics.raw_changes`, kiểm tra contract (sai thì vào `quarantine`), dedup theo định danh ổn định (bảng, LSN, transaction, operation, row image) rồi ghi `clean_rows`. Thay đổi có LSN cũ không thể ghi đè bản ghi mới hơn. Trong cùng transaction, mỗi dòng được chấp nhận còn

được ghi vào outbox `analytics.domain_outbox` để worker phát lên topic `br45.clean.v2`.



Hình 4.2. Đường CDC thật của BR4 và BR5

Ngữ nghĩa chỉ số. Mọi khoảng quan sát là nửa mở $[t_0, t_1)$. Một lần chấm test là hợp lệ khi bài nộp đã được chấm, kết quả hợp lệ và trạng thái thuộc {passed, failed, error, timeout}. Một lần làm câu hỏi là hợp lệ khi mọi test bắt buộc của phiên bản câu hỏi *tại thời điểm nộp* có kết quả hợp lệ; lần làm đó đạt khi mọi test bắt buộc đều đạt. Phiên bản test mới hơn không diễn giải lại bài nộp cũ.

$$\text{pass_rate} = \frac{\text{số lần chấm hợp lệ đạt}}{\text{tổng số lần chấm hợp lệ}}$$

Mẫu số bằng 0 cho kết quả `null` (hiển thị là chưa có số liệu). Hệ thống cũng tính pass rate ở lần làm đầu tiên và lần làm cuối cùng của mỗi người học.

Bảng 4.2. Rule phát hiện bất thường của BR4 (có thể cấu hình)

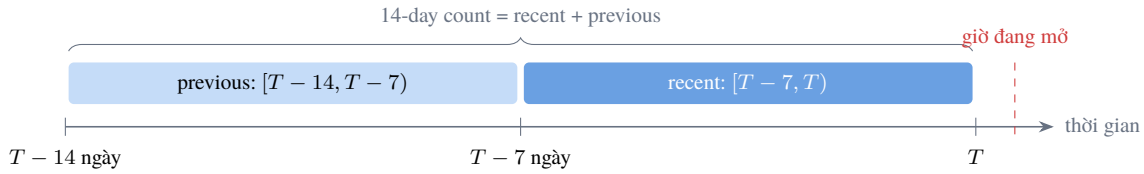
Rule	Mặc định	Trạng thái sinh ra
Số người học hợp lệ phân biệt tối thiểu	20	`INSUFFICIENT_DATA` nếu thiếu
Pass rate thấp của câu hỏi	dưới 30%	`LOW_PASS_RATE`
Pass rate thấp của test case	dưới 20%	`LOW_PASS_RATE`
Thấp hơn median của các test bắt buộc khác	$\geq$ 50 điểm %	`SUSPICIOUS_TEST_CASE`
Giảm so với window liền trước cùng độ dài	$\geq$ 30 điểm %	`NEEDS_REVIEW`

Kết quả được lưu trong ba mart của PostgreSQL (`mart_question_pass_statistics`, `mart_test_case_pass_statistics`, `mart_question_anomalies`) kèm định danh, phiên bản, window, số đếm, lý do và threshold. Phép tổng hợp chạy bất đồng bộ (Hình 4.2). Khi một thay đổi CDC của dữ liệu chấm bài được chấp nhận, hoặc khi evaluation time tiến lên, CDC worker ghi một job bền vững vào `analytics.assessment_jobs` cho window 30 ngày của từng khoá học có bài nộp. Job br45-stream nạp các dòng từ `br45.clean.v2` vào một bảng Delta bằng MERGE theo key, chỉ nhận dòng có LSN mới hơn. Khi bảng đã chứa thay đổi mà job yêu cầu, br45-stream tổng hợp bằng Spark SQL trên một snapshot có version của bảng rồi

gọi `publish()`: sau một quality gate, toàn bộ dòng mart của khoá học và window đó được thay trong một transaction PostgreSQL. Vì mart được thay bằng giá trị tuyệt đối theo key ổn định thay vì cộng dồn, việc sửa điểm hay xoá bản ghi được phản ánh đúng mà không cần chạy lại seed. API chỉ đọc mart và báo trạng thái của job, không tự chạy phép tổng hợp. Phép tổng hợp bằng SQL của PostgreSQL trên `clean_rows` vẫn được giữ làm engine đối chứng, chọn bằng `EDU_BR45_ASSESSMENT_ENGINE=postgres`. Một cờ bất thường chỉ yêu cầu giảng viên xem xét, không kết luận đề bị lỗi.

4.1.5 BR5 — Cảnh báo sớm nguy cơ dropout

Sliding window theo event time. Spark Structured Streaming đọc `br45.activity.v1`, dùng watermark 21 ngày và `dropDuplicatesWithinWatermark(event_id)`, rồi tổng hợp bằng `window(event_time, '7 days', slide)` với slide mặc định 1 giờ. Đây là stateful window aggregation, không phải một truy vấn SQL chạy theo lịch. Với evaluation time T theo giờ UTC, các feature được tính như Hình 4.3.



Hình 4.3. Các window feature tại evaluation time T (BR5)

Khi một hoạt động được cập nhật, phần đóng góp cũ được rút khỏi các window rồi phần mới được cộng vào; khi bị xoá, phần đóng góp bị rút ra. Sự kiện "clock" có trọng số 0 tạo một window rỗng để ghi nhận tình trạng không hoạt động. Mỗi micro-batch ghi đè tổng tuyệt đối của window trong một transaction. Điểm rủi ro chỉ được chấm khi offset Spark đã commit phủ hết outbox, nhờ đó tránh cảnh báo trên feature chưa đầy đủ.

Điều kiện xét. Người học phải có ghi danh hiệu lực ít nhất 7 ngày. Bài tập bắt buộc chỉ được tính khi deadline nằm trong 14 ngày gần nhất và sau ngày ghi danh; deadline trong tương lai không bị phạt. Thiếu telemetry không bị coi là không hoạt động: tín hiệu này chỉ hợp lệ khi đã từng quan sát được hoạt động thật.

Bảng 4.3. Các tín hiệu rủi ro và điểm số (BR5)

Tín hiệu	Điều kiện	Điểm
Không hoạt động	Ít nhất 7 ngày kể từ hoạt động có ý nghĩa gần nhất	35
Trễ hạn	Ít nhất một bài tập bắt buộc đã đến hạn mà chưa nộp	30
Tỷ lệ nộp thấp	Tỷ lệ nộp các bài đã đến hạn dưới 50%	20
Hoạt động giảm	Hoạt động giảm ít nhất 50%, với ít nhất 3 quan sát ở window trước	15

Điểm tối đa là 100, chia thành các mức LOW 0–29, MEDIUM 30–59, HIGH 60–79 và CRITICAL 80–100. Mọi kết quả đều kèm feature, giải thích, điều kiện xét, evaluation time và policy. **Điểm số không phải xác suất dropout**; đây là công cụ giúp cố vấn ưu tiên can thiệp.

Alert lifecycle. Mỗi cặp người học × khoá học chỉ có một alert đang mở. Vượt 60 điểm sẽ sinh alert mới; cooldown mặc định 72 giờ. Alert đang mở được cập nhật mức độ và lý do. Cố vấn có thể acknowledge hoặc resolve; hệ thống lưu thời điểm và người thực hiện. Khi điểm xuống dưới threshold, alert tự động được resolve. Lịch sử mọi lần đánh giá và sửa đổi được giữ lại để kiểm toán.

4.1.6 BR6 — Tiến độ kỹ năng theo tổ chức

BR6 là ví dụ đầy đủ nhất của pipeline medallion. Telemetry và CDC đi qua silver (migrate schema, pseudonymisation), lên gold (`fct_enrollment`, `fct_lesson_progress`, `fct_submission`) và được tổng hợp thành `mart.org_progress` theo tổ chức × khoá học. Tỷ lệ hoàn thành là trung bình tỷ lệ bài học đã hoàn thành của khoá; mức thành thạo là average pass rate của các bài nộp đã chấm, trong đó người học chưa có bài chấm được tính là 0.

k-anonymity được bảo vệ ở ba lớp. (1) Mart chỉ công bố quy mô cho nhóm dưới $k = 5$ người, các tỷ lệ để trống. (2) Quality gate mức `error` `mart_org_progress_k_anonymity` làm hỏng lần build nếu vi phạm. (3) Bộ đọc API xoá tỷ lệ của các dòng bị ẩn, ngay cả khi mart vẫn còn sót giá trị do lỗi. Quản trị viên chỉ đọc được tổ chức trong phạm vi tài khoản của mình; truy cập ngoài phạm vi trả 403.

4.1.7 BR7 — Heatmap tương tác video

Theo phiên bản. Sự kiện được quy về phiên bản nội dung có hiệu lực tại thời điểm xảy ra (`gold.dim_content` là SCD2). Video quay lại có heatmap riêng cho từng phiên bản; API mặc định chọn phiên bản mới nhất. Bảng `gold.fct_video_second_activity` có grain phiên bản

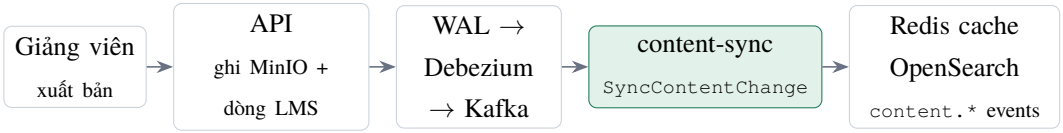
× giây × ngày, đếm play, pause, seek và số người học phân biệt. `mart.video_heatmap` tạo lưới dày theo từng giây trong 28 ngày, với số người học phân biệt chính xác và cường độ chuẩn hoá 0–1. Vị trí vượt quá độ dài video được kẹp về giây cuối.

Overlay hôm nay (ADR 0014). `telemetry-live` chạy ba truy vấn Spark có checkpoint. Truy vấn thứ nhất chuẩn hoá telemetry thô bằng `normalize_rows` của silver; hai truy vấn còn lại tổng hợp theo giây video (BR7) và điểm tương tác theo ngày (BR8), dùng watermark 5 phút và dedup theo `event_id`. Kết quả được ghi vào Redis dưới dạng giá trị tuyệt đối, nên replay ghi đè thay vì cộng thêm. API chỉ ghép overlay cho ngày sau snapshot mart và cho phiên bản mới nhất; nếu Redis lỗi, API trả dữ liệu mart.

4.1.8 BR8 — Đồng bộ nội dung và xu hướng tương tác

4.1.8.1 Đồng bộ cache và search index

Một phiên bản nội dung mới tương ứng với lần cập nhật dòng `public.course_content`, trỏ tới một đối tượng bất biến mới trong MinIO (`courses/<course>/<content>/v<n>.<ext>`). API giảng viên chỉ ghi đối tượng và dòng dữ liệu dưới row lock, tránh hai người xuất bản cùng lúc; việc đồng bộ do `content-sync` đảm nhận. Debezium bắt thay đổi; dịch vụ `content-sync` chạy use case `SyncContentChange` (Hình 4.4).



Các bước trong content-sync:
1. Lấy đối tượng 2. Vô hiệu hoá Redis cache 3. Chunk + nhúng vector 4. Dọn phiên bản khác
5. Làm mới search index 6. Làm ấm cache 7. Phát sự kiện

Hình 4.4. Đồng bộ nội dung BR8 dựa trên CDC

Idempotent theo trạng thái index, không theo thứ tự đến. `sync_policy` chỉ áp dụng phiên bản mới hơn phiên bản đã index; phiên bản bằng nhau là trùng lặp, cũ hơn là stale. Vì vậy replay CDC, khởi động lại hay backfill đều an toàn. Tìm kiếm chỉ trả các chunk thuộc phiên bản hiện hành, nên việc dọn dẹp chậm không làm lộ văn bản cũ.

Tìm kiếm lai. OpenSearch lưu chunk văn bản với chỉ mục BM25 và vector `knn_vector` (HNSW [20]); kết quả của hai phía được hợp nhất bằng reciprocal rank fusion [21] ở client. Vector hiện do một bộ nhúng băm tất định sinh ra (không cần model hay GPU), nằm sau port `Embedder`; thay bằng sentence-embedding chỉ cần một adapter mới và một lần reindex. Job Spark `edu batch content-reindex` phân phối *cùng* use case theo nhóm nội dung để backfill và ghi `silver.content_chunks`.

4.1.8.2 Xu hướng tương tác

Điểm tương tác của mỗi người học trong một window là tổng có trọng số của các hoạt động (Bảng 4.4). Mart `mart.engagement_trend` tính bốn window tuần (thứ Hai đến Chủ nhật) cho mỗi cặp người học × khoá học.

Bảng 4.4. Trọng số hoạt động trong engagement score (`EngagementWeights`)

Nhóm	Verb	Trọng số
Đăng nhập	`logged_in`	1,0
Xem bài	`viewed`, `played`, `completed`	1,0
Làm bài	`attempted`, `submitted`	2,0
Thảo luận	`posted`	1,5

Gọi a, b, c là điểm của ba window gần nhất theo thứ tự thời gian và $r = 0{,}30$ là tỷ lệ giảm. Người học được đánh dấu *đang giảm* khi

$$a > 0 \ \wedge \ b \leq a\,(1 - r) \ \wedge \ c \leq b\,(1 - r),$$

tức là điểm giảm ít nhất 30% trong *mỗi* window của hai tuần liên tiếp. Overlay "hôm nay" dùng cùng bộ trọng số, nhờ đó giá trị trong ngày khớp với giá trị mà data warehouse tính vào hôm sau.

Pipeline chỉ có giá trị khi kết quả đến được người dùng. Mục 4.2 trình bày ứng dụng web cho năm nhóm: người học, giảng viên, cố vấn học tập, quản trị viên tổ chức và đội vận hành nền tảng dữ liệu.

4.2 Giao diện người dùng

4.2.1 Quy trình thiết kế

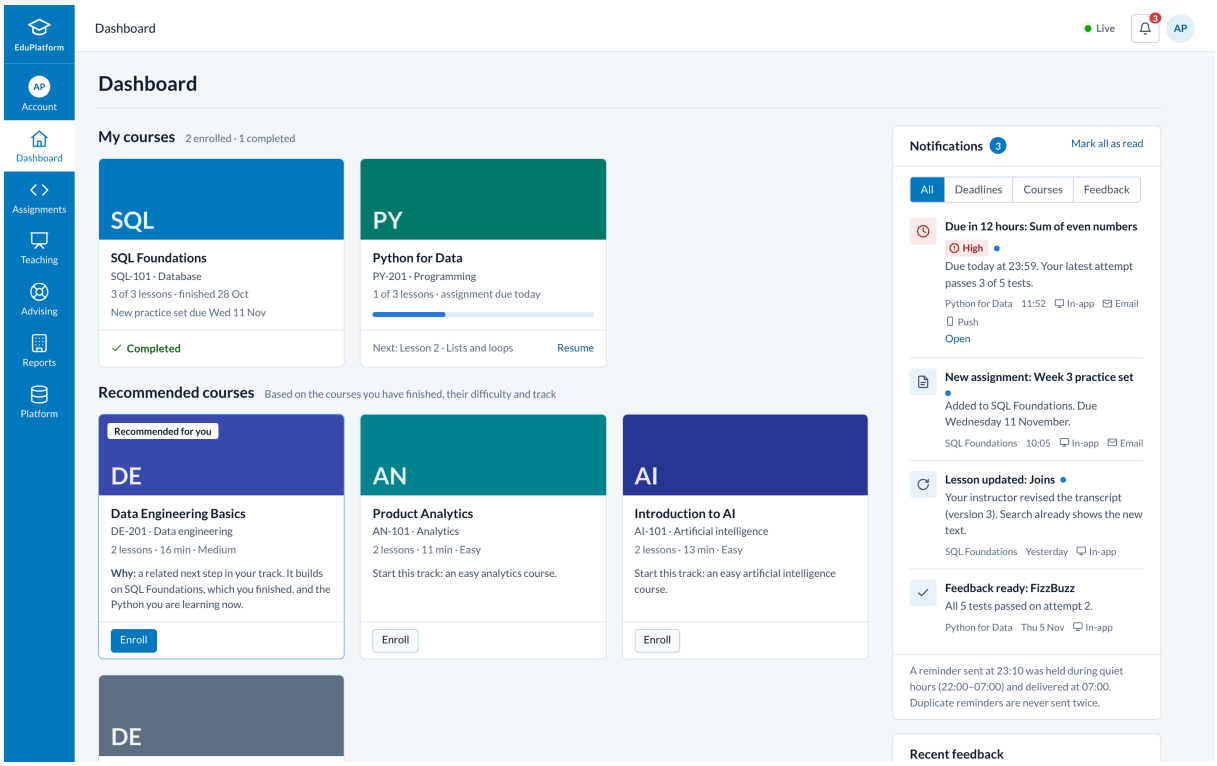
Giao diện trải qua ba vòng thiết kế:

1. **Dashboard hướng nhà phát triển**: nút giả lập sự kiện, mã định danh thô và ô nhập token nằm lẫn với kết quả. Bản này chứng minh API hoạt động nhưng khó dùng với người dùng cuối.
2. **Mockup HTML tương tác và bản React đầu tiên**: mockup dùng dữ liệu mẫu nhưng tuân theo rule nghiệp vụ thật (điểm rủi ro 35/30/20/15, threshold 30/60/80, $k = 5$, tỷ lệ giảm 30%), nhờ đó nhóm thống nhất được bố cục trước khi viết mã. Khi xem lại, nhóm

nhận thấy bản này mang dáng dấp của một dashboard chung chung (thẻ bo tròn lớn, nền chuyển màu, nhãn in hoa) hơn là một hệ thống quản lý học tập.

3. **Thiết kế lại theo quy ước LMS**: dựa trên Canvas và Moodle [22, 23], dùng thanh điều hướng toàn cục bên trái, menu theo khu vực, breadcrumb, thẻ khoá học có dải màu và trang bài tập có dòng thông tin chi tiết; chế độ sáng tông xanh da trời là mặc định. Mockup dùng cùng bộ design token với ứng dụng (CSS được sao chép và giữ đồng bộ), nên hai bản thống nhất về ngôn ngữ thị giác (Hình 4.5).

Ứng dụng được viết bằng React 18 + TypeScript trên Vite, không thêm thư viện UI hay chart; mọi màn hình đọc API thật. Những phần của mockup không có API tương ứng (độ tươi của báo cáo, kết quả kiểm tra chất lượng, ghi chú can thiệp) được bỏ thay vì dùng dữ liệu giả.



Hình 4.5. Mockup: Dashboard người học với thẻ khoá học (BR2), thông báo (BR1) và phản hồi gần đây (BR3)

4.2.2 Đáp ứng yêu cầu về giao diện

Đề bài yêu cầu kết quả của mỗi BR được hiển thị phù hợp với ngữ cảnh miền ứng dụng, với đặc điểm và thói quen của nhóm người dùng đích, đồng thời hài hoà cả về trình bày lẫn tương tác.

Ngữ cảnh miền ứng dụng. Người học và giảng viên đã quen với các LMS như Canvas và Moodle, nên ứng dụng dùng lại bố cục và từ vựng của chúng thay vì đặt ra một kiểu điều hướng mới. Cụ thể: thanh điều hướng toàn cục màu xanh da trời ở cạnh trái, mỗi mục gồm biểu tượng

và nhãn (Account, Dashboard, Assignments, Teaching, Advising, Reports, Platform); menu chữ theo khu vực, ẩn hoặc hiện bằng nút ba gạch; breadcrumb cho biết vị trí hiện tại, ví dụ "Teaching › Assessment health". Dashboard của người học đặt thẻ khoá học có dải màu, mã khoá và tiến độ; khoá đề xuất (BR2) hiện thành thẻ có nhãn "Recommended for you" và dòng "Why" giải thích lý do; cột phải chứa Notifications (BR1) và Recent feedback (BR3), tương tự khối To Do và Recent Feedback của Canvas. Trang bài tập (BR3) theo mẫu trang Assignment: tên khoá học, tên bài, dòng "Language · Tests · Attempts · Submitting", phần Instructions, khung phản hồi và Submission history. Các nhãn BR1–BR8 phục vụ truy vết khi trình bày nhưng không có trong LMS thật, nên mặc định tắt và có thể bật lại ở Account › Preferences.

Đặc điểm người dùng đích. Mỗi nhóm có thiết bị, nhịp làm việc và câu hỏi riêng; Bảng 4.5 ánh xạ những đặc điểm đó vào các lựa chọn thiết kế cụ thể.

Bảng 4.5. Đặc điểm người dùng và lựa chọn thiết kế tương ứng

Nhóm	Đặc điểm, thói quen	Lựa chọn thiết kế
Người học	Học trên cả điện thoại lẫn máy tính, session ngắn; cần biết ngay việc tiếp theo và lý do của mỗi gợi ý	Thẻ khoá học, deadline và thông báo trên cùng một trang; gợi ý luôn kèm lý do; bố cục co về 390 px, thanh bên thành dải ngang; trang bài tập tách rõ đề bài, mã và phản hồi từng test
Giảng viên	Làm việc trên máy tính, quản lý nhiều khoá; xem phân tích theo từng khoá và từng bài	Trang phân tích nằm trong khu Teaching có menu riêng; bộ lọc khoá học ở đầu trang; bảng có hàng chọn được để xem chi tiết; heatmap đánh dấu A, B, C các thời điểm cần xem lại
Cố vấn	Xử lý queue theo mức ưu tiên; cần biết vì sao một người học bị gắn cờ trước khi liên hệ	Queue xếp theo điểm rủi ro; điểm phân rã thành 35 + 30 + 20 + 15 kèm câu giải thích; nút Acknowledge và Resolve theo alert lifecycle
Quản trị tổ chức	Đọc báo cáo định kỳ, chuyển tiếp cho cấp trên; quan tâm quyền riêng tư	Ô số liệu tổng hợp, bảng theo khoá học và nút Export CSV; nhóm dưới 5 người bị ẩn kèm thông báo giải thích, không hiện tỷ lệ hay quy mô chính xác
Đội nền tảng	Theo dõi sức khoẻ pipeline trong lúc vận hành	Bộ đếm từng chặng ingest → bronze → silver → đầu ra, dead-letter queue và luồng sự kiện trực tiếp

Hài hoà trong trình bày và tương tác. Một màu nhấn duy nhất là xanh da trời `#0277BD`, dùng cho thanh điều hướng, liên kết, nút chính và mục đang chọn; dải màu tuần tự của heatmap và chuỗi dữ liệu thứ nhất của biểu đồ cũng thuộc họ xanh dương, nên biểu đồ và khung giao diện

cùng tông. Chế độ sáng là mặc định, kể cả khi hệ điều hành đang để chế độ tối, vì người dùng LMS thường đọc lâu và học vào ban ngày; chế độ tối là tuỳ chọn trong Account › Preferences, dùng nền xanh navy với cùng màu nhấn. Phong cách phẳng và nhất quán: font Lato, góc bo 4–6 px, khối nội dung viền mảnh, không bóng đổ, không nền chuyển màu.

Màu trạng thái (tốt, cảnh báo, nghiêm trọng, nguy hiểm) chỉ dùng cho trạng thái và luôn kèm biểu tượng cùng nhãn chữ; bảng màu biểu đồ đã được kiểm định khả năng phân biệt với người mù màu trên cả nền sáng lẫn nền tối; các cặp chữ và nền chính đạt tương phản tối thiểu 4,5:1 theo WCAG 2.1 AA [24], kể cả chữ trắng trên dải màu của thẻ khoá học. Tương tác theo một mẫu chung ở mọi trang: bộ lọc ở đầu trang, các ô số liệu tổng hợp, danh sách hoặc bảng, rồi chi tiết của hàng đang chọn; hàng được chọn có nền nhạt và vạch màu nhấn bên trái, mọi điều khiển có viền focus khi dùng bàn phím.

4.2.3 Design system

- **Design token** cho màu, khoảng cách, bo góc và bóng đổ, có bộ giá trị riêng cho chế độ sáng và tối; chế độ sáng là mặc định, chế độ tối chuyển ở Account › Preferences.
- **Khả năng tiếp cận**: mọi biểu đồ có tooltip và bảng "Xem dạng bảng" thay thế; nút có nhãn cho trình đọc màn hình; bố cục co giãn tới điện thoại mà không cuộn ngang.
- **Nhãn yêu cầu**: mỗi trang và khối được gắn nhãn BR, mặc định tắt và có thể bật lại ở Account › Preferences để truy vết giao diện về yêu cầu khi trình bày.

4.2.4 Kiến trúc frontend

Ứng dụng được tổ chức theo *workspace*, mỗi nhóm người dùng một workspace chọn ở thanh bên (Bảng 4.6). Thanh trên cùng có chỉ báo sự kiện trực tiếp (Server-Sent Events), thông báo người học và trạng thái đăng nhập; mục Account mở menu cho biết vai trò đang đăng nhập và chứa tuỳ chọn chế độ tối cùng nhãn yêu cầu.

Bảng 4.6. Workspace, route và trang

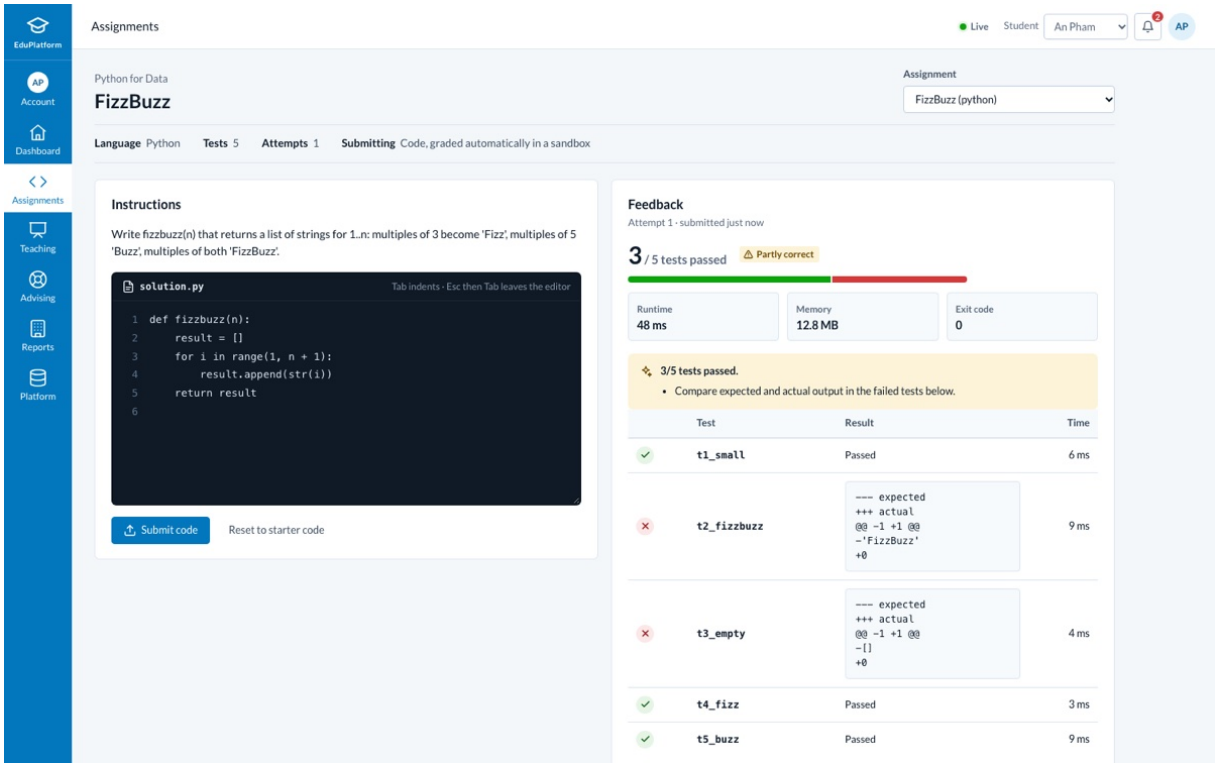
Workspace	Route	Nội dung	BR
Learner	`/learner`	Khoá học, gợi ý, phản hồi gần đây, thông báo	1, 2
Learner	`/learner/code-lab`	Trình soạn mã, phản hồi từng test, các lần nộp	3

Workspace	Route	Nội dung	BR
Instructor	`/instructor/assessments`	Câu hỏi và chi tiết test case	4
Instructor	`/instructor/videos`	Heatmap theo phiên bản, overlay hôm nay, thời điểm cần xem lại	7
Instructor	`/instructor/engagement`	Trung bình khoá học, người học đang giảm	8
Instructor	`/instructor/content`	Xuất bản phiên bản, theo dõi đồng bộ, xem trước tìm kiếm	8
Advisor	`/advisor`	Queue rủi ro, phân rã điểm, lịch sử, cảnh báo	5
Org admin	`/admin`	Tiến độ tổ chức có ẩn nhóm nhỏ, xuất CSV	6
Data platform	`/pipeline`	Bộ đếm luồng, throughput, topic, DLQ, silver	–

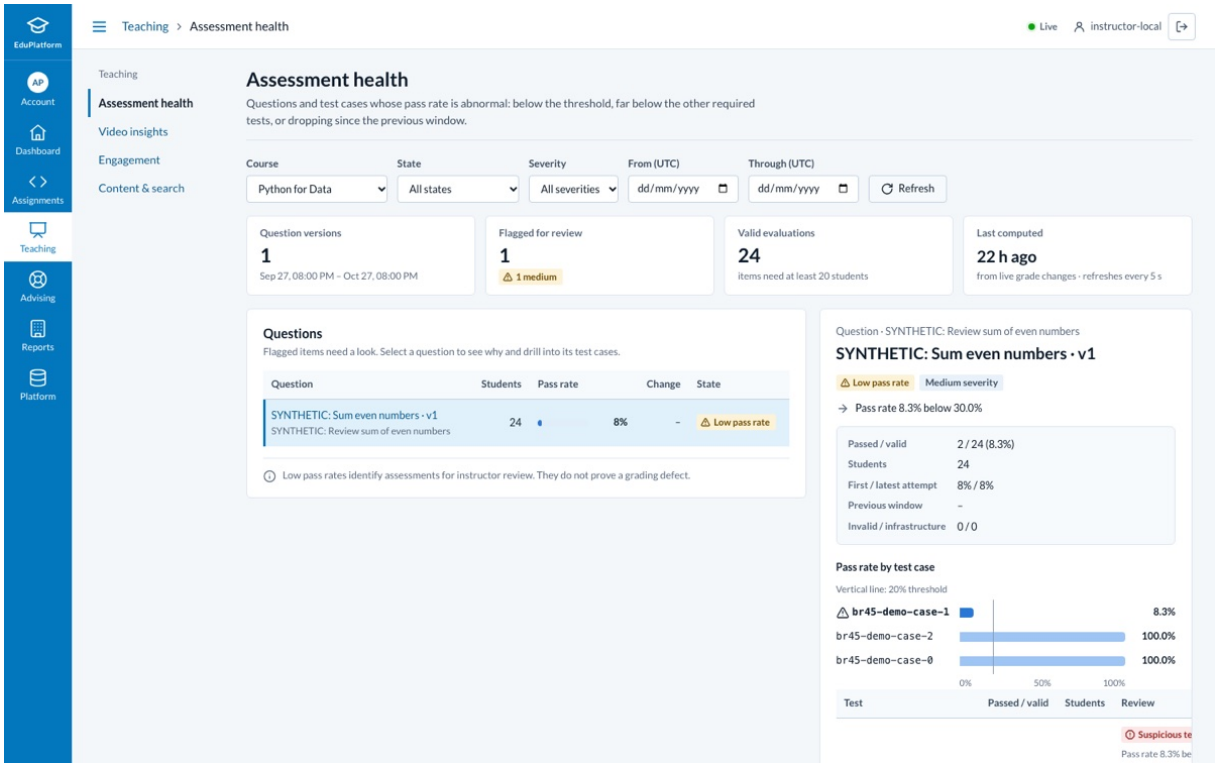
Workspace Instructor, Advisor và Org admin được bảo vệ bằng thẻ đăng nhập bằng tên người dùng và mật khẩu, mỗi workspace một session riêng. Máy chủ đối chiếu mật khẩu với hash Argon2id của các tài khoản cấu hình sẵn (`EDU_STAFF_ACCOUNTS`) rồi cấp cookie session HttpOnly theo vai trò, có hạn tám giờ; các request sau đó gửi cookie này kèm header `X-Staff-Role`. Danh tính, vai trò và phạm vi khoá học/tổ chức được lấy từ tài khoản phía máy chủ, giao diện không tự khai báo quyền. Các điều khiển thử nghiệm (giả lập sự kiện LMS, bộ sinh telemetry) được tách vào khối "Demo controls" viền nét đứt. Biểu đồ được vẽ bằng SVG thuần theo chiều rộng thật của phần tử, nên chữ không bị co giãn.

4.2.5 Các màn hình chính

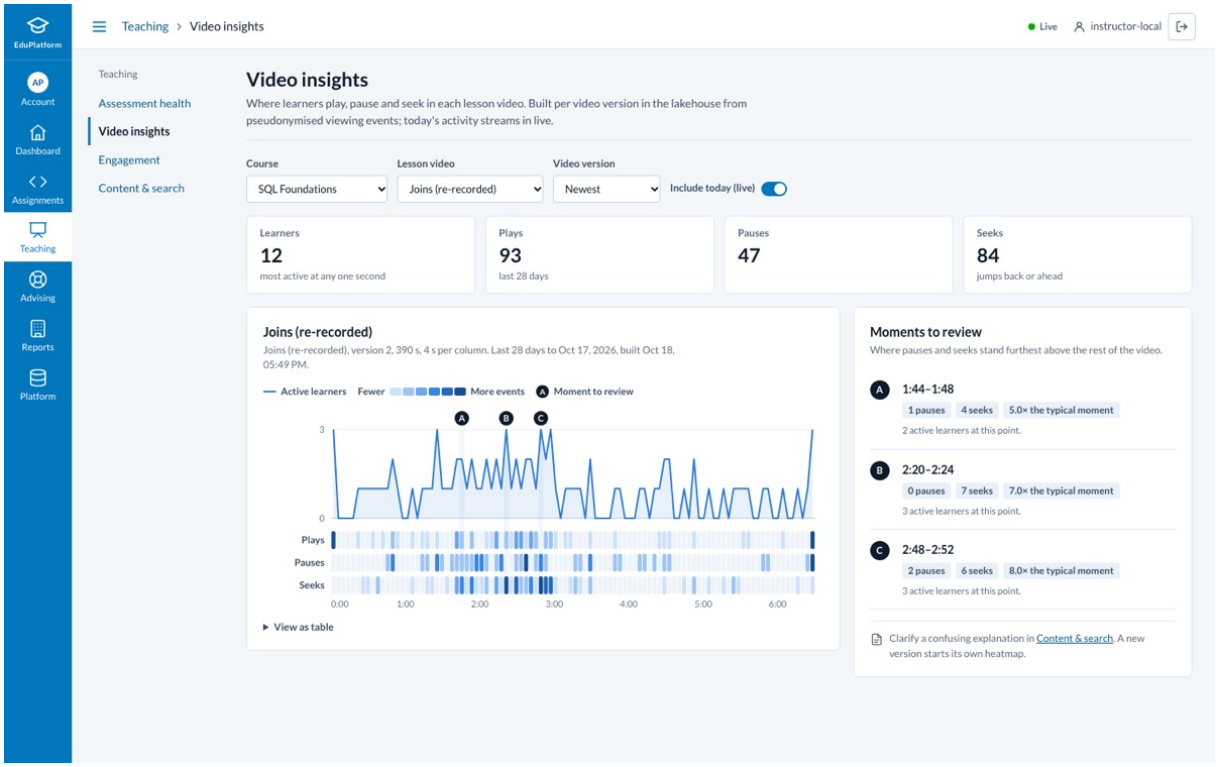
Các ảnh trong mục này được chụp từ ứng dụng thật trên dữ liệu hệ thống, ở chế độ sáng mặc định, trừ khi được ghi rõ là mockup.



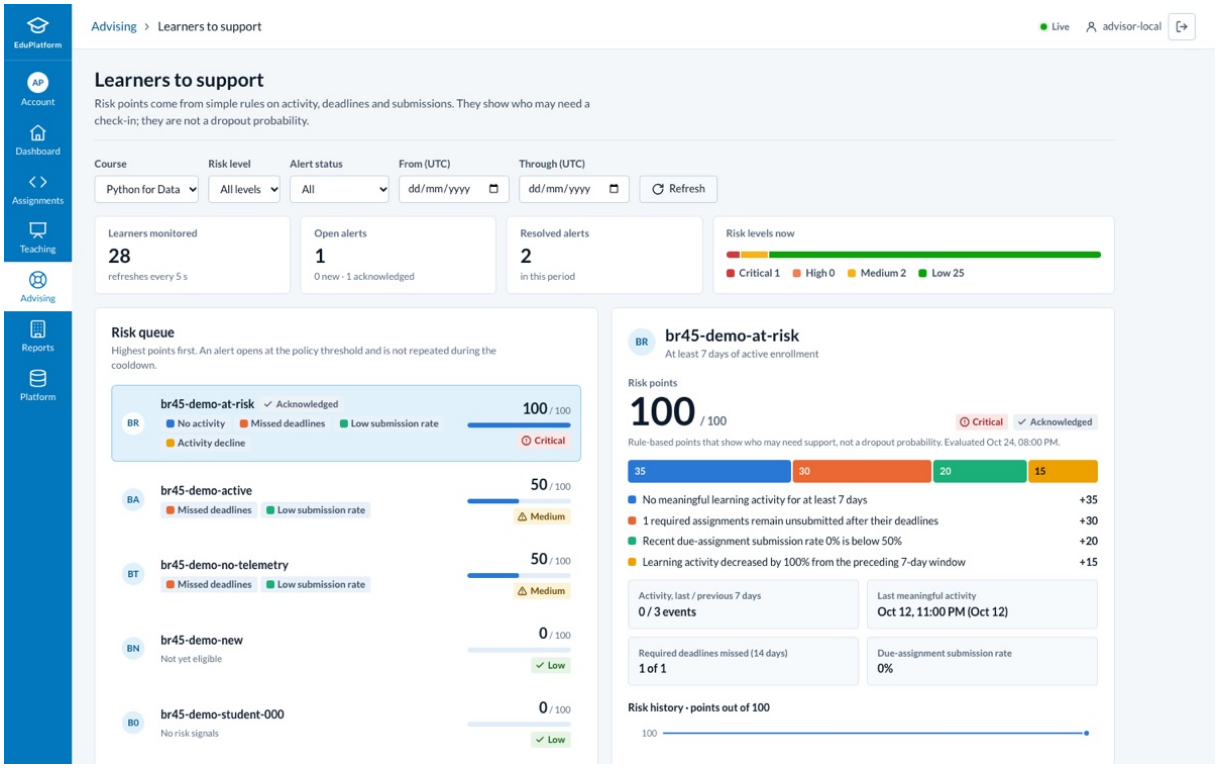
Hình 4.6. Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff



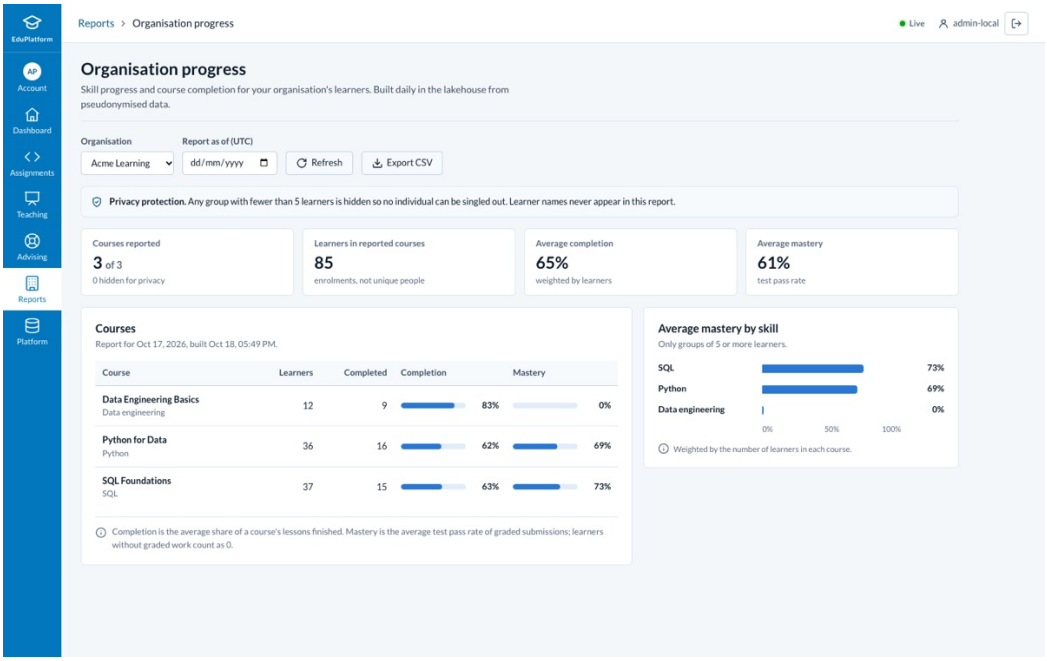
Hình 4.7. Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20%



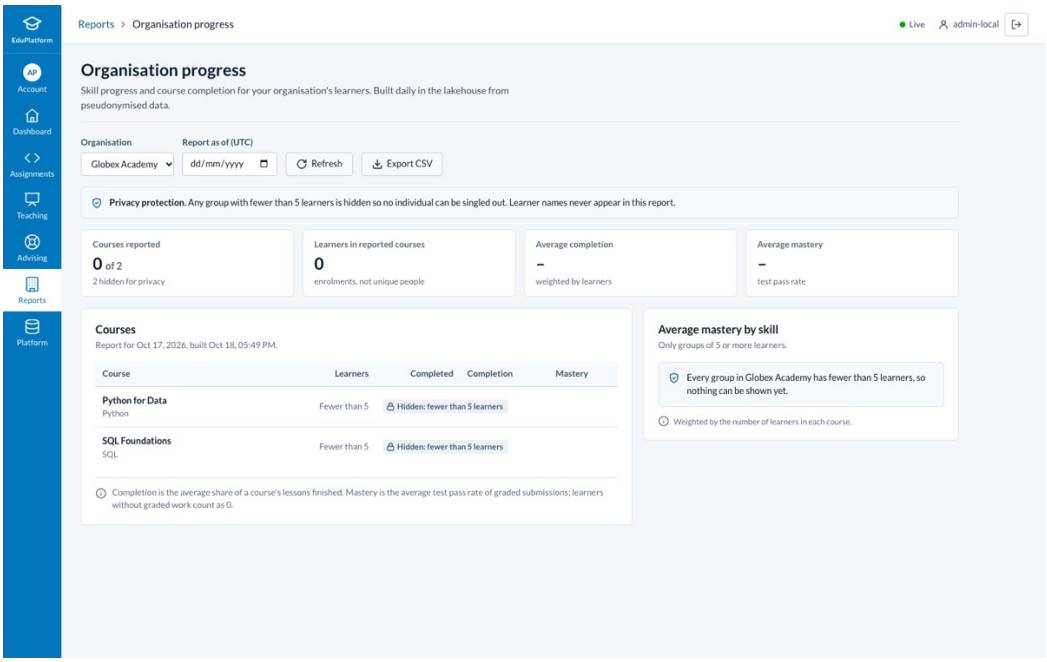
Hình 4.8. Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video "Joins" (390 giây), kèm các thời điểm cần xem lại



Hình 4.9. Learners to support (BR5): queue rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15

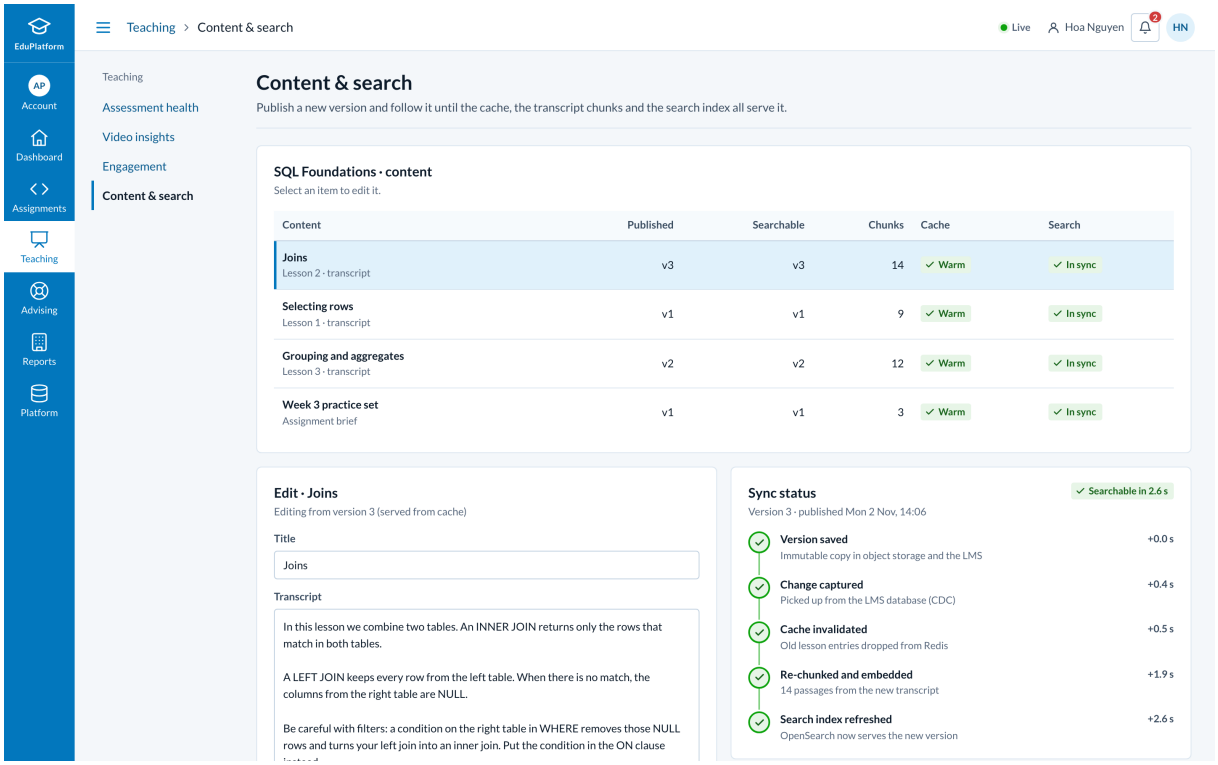


(a) Acme Learning: mọi nhóm đủ 5 người nên được báo cáo

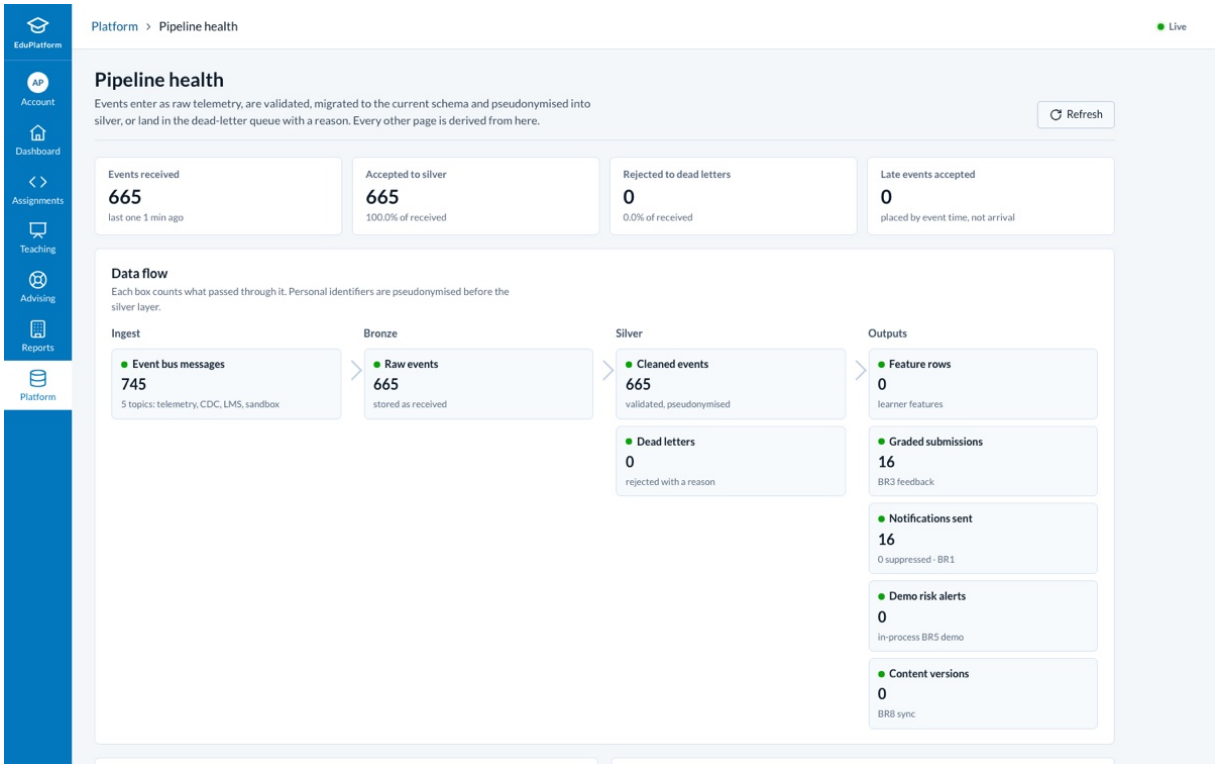


(b) Globex Academy: mọi nhóm dưới 5 người nên bị ẩn, không lộ tỷ lệ hay quy mô chính xác

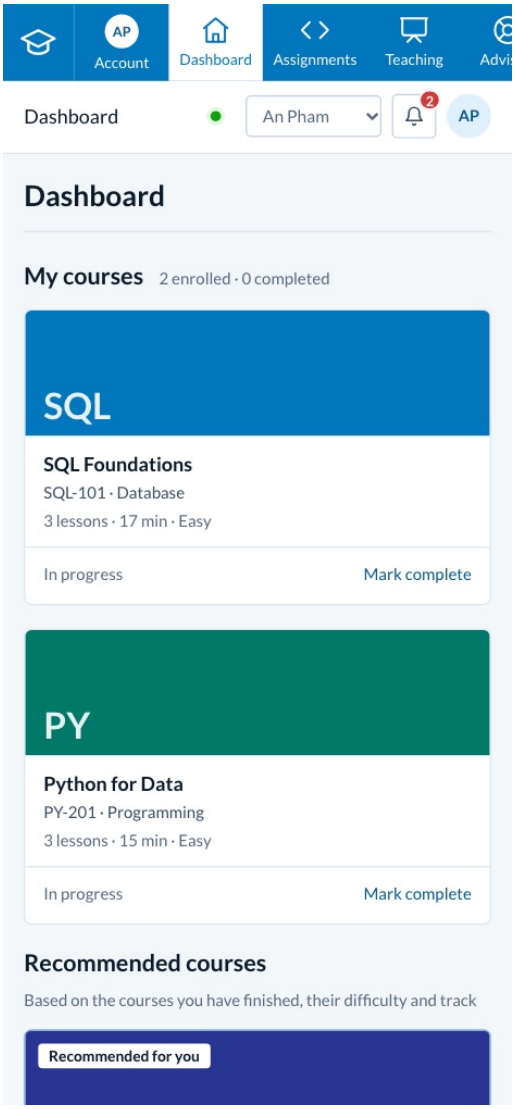
Hình 4.10. Organisation progress (BR6): k-anonymity với $k = 5$



Hình 4.11. Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được



Hình 4.12. Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu



Hình 4.13. Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang

4.2.6 Kiểm thử giao diện

Giao diện được kiểm chứng theo ba cách:

- **Biên dịch nghiêm ngặt**: TypeScript `strict`; bản build production thành công.
- **Kiểm tra thủ công trên trình duyệt** với dữ liệu thật ở chế độ sáng/tối, desktop và điện thoại: một bài nộp đi qua sandbox sinh thông báo mới; các trang BR6–BR8 đọc mart thật từ Trino.
- **Acceptance test bằng Playwright** trên stack thật, gồm 12 test trong năm tệp spec: 2 test BR1–BR3, 3 test BR4/BR5, 4 test BR6–BR8, 2 test cho hai chế độ classroom và replay của BR4/BR5 và 1 test session đăng nhập nhân sự. Chín test của ba bộ đầu đã đạt ở các lần chạy trước khi gộp nhánh BR4/BR5; kết quả JSON và ảnh chụp được lưu kèm. Trên mã đã gộp, các test này đăng nhập bằng tài khoản nhân sự và cả bộ 12 test chưa được chạy

lại.

Chương 5

Kiểm thử và đánh giá

Chương này đánh giá tính đúng đắn và hiệu năng của hệ thống. Mọi số liệu được đo trên dữ liệu tổng hợp, trên một máy cá nhân chạy stack Docker Compose. Do giới hạn bộ nhớ, mỗi phép đo chỉ bật những dịch vụ cần thiết (Mục 5.3); đây là quan sát cục bộ, không phải cam kết về năng lực hệ thống. Các tệp CSV gốc lưu mọi mẫu đo (Phụ lục A).

5.1 Chiến lược kiểm thử

Chiến lược kiểm thử được chia thành nhiều tầng, từ domain thuần đến dịch vụ thật và trình duyệt; Bảng 5.1 tóm tắt số lượng và phạm vi của từng tầng. Số lượng được đếm trên mã đã gộp nhánh BR4/BR5; trên mã này, đợt cập nhật báo cáo chỉ chạy lại tầng unit, nên con số của các tầng còn lại phản ánh số test hiện có chứ không phải kết quả chạy.

Mã Python được kiểm tra bằng `ruff` và `mypy`; data warehouse chạy đủ 10 quality gate sau mỗi lần build.

Unit test tập trung vào policy và use case; integration test dùng PostgreSQL, Kafka và Redis thật; browser test chạy trên stack đầy đủ. Nhờ cách phân tầng này, các tầng dưới vẫn cho phản hồi nhanh, còn luồng end-to-end vẫn được kiểm chứng.

Bảng 5.1. Các tầng kiểm thử

Tầng	Số lượng	Phạm vi
Unit	379	Policy và use case của domain, phân quyền API với adapter giả, session đăng nhập nhân sự, phân tích bản ghi Debezium, S3 (moto), Redis (fakeredis), OpenSearch adapter, overlay, cache; architecture test kiểm tra ranh giới các lớp
Spark	11	SQL của bộ đọc mart trên bảng Delta dựng từ DDL; job reindex trên Delta với S3 giả lập; tính tương đương của streaming overlay trên Kafka và Redis thật; BR4: sửa điểm trên Delta giữ snapshot cũ và khớp phép tính đối chứng; BR5: checkpoint window dedup sau khi khởi động lại
Integration BR4/BR5	21	PostgreSQL và API: SQL khớp Python, lần làm đầu/cuối, kết quả thiếu hoặc không hợp lệ, lỗi hạ tầng và lỗi code, đổi phiên bản, LSN cũ, cách ly late data, phân quyền, alert lifecycle, lease của job tổng hợp, phiên bản policy, ghi chú can thiệp, retention
Integration BR1/BR3	1	HTTP end-to-end: enroll, nộp bài, chờ `graded`, kiểm tra thông báo từ `grading.submission_graded.v1`
Dịch vụ thật	–	OpenSearch 2.17 (analyzer, HNSW, lọc kNN, chỉ phiên bản hiện hành); catalog PostgreSQL (5 người xuất bản đồng thời, rollback); tính tương đương của overlay kể cả gửi lại và replay toàn bộ
End-to-end	1 + 6	Luồng PostgreSQL → WAL → Debezium → Kafka → Spark → API với 6 kiểm tra chấp nhận, gồm sửa điểm, replay Kafka, UPDATE/DELETE hoạt động và khởi động lại checkpoint
Trình duyệt	12	Playwright: 2 test BR1–BR3, 3 test BR4/BR5, 4 test BR6–BR8, 2 test cho chế độ classroom và replay của BR4/BR5 và 1 test session đăng nhập nhân sự; drill-down của giảng viên và cập nhật khi sửa điểm, xác nhận cảnh báo và hồi phục rủi ro, lỗi đăng nhập, k-anonymity, heatmap theo phiên bản, xuất bản đến khi tìm kiếm được, overlay trực tiếp

5.2 Kiểm chứng nghiệp vụ

Mỗi BR có ít nhất một kịch bản kiểm tra rule nghiệp vụ và được kiểm chứng tới tận giao diện.

Bảng 5.2. Các kịch bản kiểm chứng nghiệp vụ và kết quả

BR	Kịch bản	Kết quả quan sát
1	Đăng ký `l-002` vào `c-ml-301`, gửi deadline reminder, replay cùng sự kiện	PostgreSQL có enrollment; Redis inbox có đúng một thông báo và `count=1`; lần replay trả `duplicate`, không tăng counter; trong đợt kiểm chứng, email và webhook đều được gửi thành công
2	5 sự kiện telemetry live, thêm 2 sự kiện, chạy lại; build warehouse rồi materialise	Session có 5 rồi 7 sự kiện, cùng `started_at`; replay không đếm lại; Redis có 40 key longterm; API trả cả hai nhóm feature
3	Nộp bài qua BFF; replay queue/raw result; bật projector	`queued` → `running` → `graded`, pass rate 0,8; `grading.*` có dòng; inbox có thông báo; replay không tạo grade thứ hai; projector 9/9 → 11/11 rồi trả baseline
4	24 người học, 3 test bắt buộc; 2 người đạt hết, 22 người trượt cùng một test	Câu hỏi đạt $2/24 = 8{,}33\%$; hai test đạt 100%, test lỗi đạt 8,33% và thấp hơn median 91,67 điểm %: gắn cờ `SUSPICIOUS_TEST_CASE`. Sửa một kết quả trong PostgreSQL làm số hiển thị đổi từ 8% sang 13% mà không tải lại trang.
4	Data warehouse: test hỏng `t5_pairs` và test khó nhưng hợp lệ `t3_ties`	Chỉ `t5_pairs` (0/68 lượt đạt) bị gắn cờ; `t3_ties` (0,55) không bị gắn cờ
5	Người học có 4 hoạt động tuần trước, không hoạt động tuần này, một bài đến hạn chưa nộp	100 điểm, mức CRITICAL, sinh cảnh báo. Bốn hoạt động thật ghi qua API làm điểm giảm còn 50 và cảnh báo tự resolve. Replay Kafka không sinh cảnh báo trùng.
5	Data warehouse: rủi ro theo ngày của một người học	Một cảnh báo khi vượt threshold; người học bỏ học 6 ngày sau, nhãn kết quả `dropped_within_30d = true`
6	Globex có các nhóm 2 người; Acme có nhóm lớn	Globex chỉ công bố quy mô; Acme được báo cáo; truy cập ngoài phạm vi trả về 403
7	Video `v-sql-joins` được quay lại	Hai heatmap riêng: v1 (420 giây) và v2 (390 giây)
8	Xuất bản transcript có chuỗi đánh dấu duy nhất	Phiên bản mới tìm kiếm được, cache trả `miss` rồi được làm ấm; xoá qua CDC gỡ vector và mục tìm kiếm

BR	Kịch bản	Kết quả quan sát
–	Chạy data warehouse: backfill, hai lần incremental, rồi chạy lại	Fingerprint các bảng giống hệt sau lần chạy lại (idempotent); một lần đồng bộ mobile muộn tự kéo window gold lùi hai ngày

Ba kịch bản BR1–BR3 chạy trên stack thật với Kafka, Redis, PostgreSQL và Spark: integration test đạt và Playwright 2/2 đạt. Trên mã đã gộp nhánh BR4/BR5, cả 379 unit test đều đạt, không có test nào bị bỏ qua.

5.3 Môi trường đo

Toàn bộ phép đo được thực hiện ngày 13/11/2026 trên một máy Apple M1 Pro (10 nhân CPU: 8 nhân hiệu năng và 2 nhân tiết kiệm điện), 32 GB RAM, macOS 26.6.2. Docker Desktop (Docker Engine 29.7.2) được cấp 10 CPU và 8 GB RAM (máy ảo báo 7,75 GiB), kèm 1 GiB swap; container Hive Metastore dùng image amd64 nên chạy giả lập trên ARM. Stack Docker Compose đầy đủ (`docker-compose.yml` và `docker-compose.br13.yml` với các profile `br45`, `br678`, `airflow`, `pipelines`) gồm 30 container chạy lâu dài: Kafka, Kafka UI, Schema Registry, Debezium, PostgreSQL, Redis, MinIO, Hive Metastore cùng cơ sở dữ liệu của nó, Trino, Flink (jobmanager, taskmanager), Spark (master, worker), OpenSearch, Airflow (webserver, scheduler, metadata database), API demo, các dịch vụ notification, grading (ingest, worker, normalizer), push-sink, Mailpit, content-sync, telemetry-live, lakehouse-streaming, br45-cdc và br45-stream.

Các phép đo chia thành hai đợt trong cùng ngày, trên cùng máy và cùng cấu hình Docker. Đợt đầu đo toàn bộ BR1–BR8 và phần so sánh Debezium với polling trên mã trước khi gộp nhánh BR4/BR5 (commit `4d70703`). Sau đó nhánh `Benchmark_BR45` được gộp vào `main` (commit `05bb5af`); lần gộp này thay đổi đúng những đường mà BR4 và BR5 đo, nên hai phép đo này được chạy lại trên mã đã gộp: BR4 từ 21:17:33 đến 21:18:13 và BR5 từ 21:19:23 đến 21:20:42 ngày 13/11/2026 (giờ Việt Nam, UTC+7). Mọi số liệu của hai thí nghiệm quy mô BR4 và BR5 trong chương này lấy từ lần đo lại đó. Các phép đo còn lại (BR1–BR3, BR6–BR8, warehouse build, Debezium và polling) giữ kết quả của đợt đầu trên commit `4d70703` và chưa được đo lại trên mã đã gộp.

Với 8 GB, stack đầy đủ không chạy ổn định: trong 25 phút đầu của đợt đo đầu, khi thử các cấu hình 24–27 container, bộ nhớ khả dụng của máy ảo nhiều lần về gần 0 và Trino bị dừng đột ngột rồi tự khởi động lại 14 lần (dấu hiệu của OOM-kill). Vì vậy mỗi phép đo chạy trên một cấu hình rút gọn, và kết quả chỉ được giữ khi cấu hình đó ổn định trong suốt measurement window. Thứ tự tạm dừng là: trước hết các job streaming mà phép đo không dùng (lakehouse-streaming, br45-

cdc, br45-stream, telemetry-live); nếu vẫn thiếu bộ nhớ thì đến các dịch vụ nhàn rỗi không nằm trên đường đo (cụm Flink chưa có job, Kafka UI), rồi đến Airflow (mọi DAG đang tạm dừng); riêng warehouse build còn phải dừng OpenSearch và content-sync. Bảng 5.3 ghi cấu hình của từng phép đo; hàng BR4/BR5 là cấu hình của lần đo lại trên commit `05bb5af`, các hàng còn lại thuộc đợt đầu trên commit `4d70703`.

Trạng thái mọi container được chụp lại ngay trước và ngay sau mỗi phép đo. Một lần đo chỉ được giữ khi tập container đang chạy, số lần restart và thời điểm khởi động của từng container không đổi trong suốt measurement window; các lần đo có container bị dừng giữa chừng đều bị loại (cột ghi chú của Bảng 5.3). Bảng 5.4 tóm tắt môi trường cho BR4–BR8.

Bảng 5.3. Container chạy trong từng phép đo (stack đầy đủ: 30 container)

Phép đo	Chạy	Tạm dừng trong lúc đo	Ghi chú
BR1–BR3; BR6–BR8 serving mart; BR8 xuất bản; Debezium và polling	26	lakehouse-streaming, br45-cdc, br45-stream, telemetry-live	Chỉ tạm dừng các job streaming mà phép đo không dùng
BR4; BR5 (đo lại trên commit `05bb5af`)	23	lakehouse-streaming, telemetry-live, Flink (jobmanager, taskmanager), Kafka UI, Airflow (scheduler, webserver)	Cùng tập container như lần đo BR4/BR5 của đợt đầu (khi đó Airflow chạy cùng br45-stream chỉ để lại khoảng 560 MiB bộ nhớ khả dụng nên được tạm dừng). Ở lần đo lại, bộ nhớ khả dụng lúc bắt đầu đo là khoảng 740 MiB (BR4) và 590 MiB (BR5), swap 1 GiB đã đầy
BR7/BR8 overlay	23	lakehouse-streaming, br45-cdc, br45-stream, Flink (jobmanager, taskmanager), Airflow (scheduler, webserver)	Hai lần đo khi Airflow còn chạy (27 và 24 container) bị loại vì Trino khởi động lại giữa chừng
Warehouse build	19	như overlay, thêm Kafka UI, OpenSearch, content-sync, telemetry-live	Hai lần thử với 24 và 22 container bị loại vì tiến trình Spark bị dừng giữa chừng (dấu hiệu của OOM-kill)

Bảng 5.4. Môi trường đo hiệu năng

	BR4 / BR5 (13/11/2026, đo lại trên commit `05bb5af`)	**BR6–BR8 (13/11/2026, commit `4d70703`)**
Máy	Apple M1 Pro (10 nhân), 32 GB, macOS 26.6.2; Docker Desktop: 10 CPU, 8 GB (7,75 GiB)	Apple M1 Pro (10 nhân), 32 GB, macOS 26.6.2; Docker Desktop: 10 CPU, 8 GB (7,75 GiB)
Engine	Spark 3.5.5 và Delta 3.3.2 (image `de261/spark`), `local[2]`, driver 1 GiB, state store RocksDB, container giới hạn 2 CPU và 2.304 MiB; PostgreSQL 16.15; script đo, phép tổng hợp Python (BR4) và phép tính batch (BR5) chạy ngoài Docker bằng Python 3.13	Spark 3.5.5, Delta 3.3.2 (warehouse build: `local[*]`, 10 luồng); Trino 478 (heap 1,2 GB); OpenSearch 2.17.1 (heap 512 MB)
Lặp lại	3 lần mỗi kích thước	2–30 lần tuỳ thí nghiệm
Dữ liệu	Tổng hợp	Tổng hợp

Ở lần đo lại BR4/BR5, hai image `de261/demo` và `de261/spark` được build lại từ commit `05bb5af`, và các migration bổ sung của schema `analytics` được áp dụng trước khi đo. Job br45-stream chạy đúng cấu hình Compose mặc định của mã đã gộp: Spark 3.5.5 `local[2]` trong container giới hạn 2 CPU và 2.304 MiB, và mã nguồn tự gắn thêm fingerprint cấu hình `ff4b0f4a118a` vào thư mục checkpoint mặc định của Compose (`br45-v2`). Tệp Compose override ngoài repo dùng ở đợt đầu để đổi thư mục checkpoint nay không còn cần thiết. Vì đây là checkpoint mới, job đọc lại topic `br45.activity.v1` từ offset sớm nhất mà Kafka còn giữ (3394); phép đo chỉ bắt đầu sau khi job đã bắt kịp cuối topic và các job tổng hợp BR4 trên Spark do lần seed sinh ra đã hoàn tất.

5.4 Kết quả hiệu năng

5.4.1 BR1–BR3: end-to-end latency

Ba đường BR1–BR3 được đo ngày 13/11/2026 trên stack nói trên (26/30 container, Bảng 5.3), sau khi loại các mẫu khởi động nóng. Mỗi số đo là latency của một chặng cụ thể sau khi stack đã dựng xong và các dịch vụ đã ấm, không tính thời gian khởi động stack. BR1 đo từ lúc gửi sự kiện LMS đến khi thông báo đọc được qua API; BR2 đo thời gian phản hồi của API gợi ý có feature store trực tuyến; BR3 đo thời gian nhận 202 và từ lúc nộp đến khi trạng thái chuyển thành

`graded`. Mỗi mẫu BR1/BR3 lặp 20 lần, BR2 lặp 30 lần, và cả phép đo lặp năm lượt liên tiếp; bảng gộp cả năm lượt (n = 100 cho BR1/BR3, n = 150 cho BR2). Trước mỗi mẫu BR1, dedup key và inbox của người học được xoá: API trả toàn bộ inbox, nên nếu inbox tích luỹ thì phép đo sẽ chậm dần và không còn phản ánh latency của chặng. Trạng thái chấm bài trong Redis cũng được xoá trước mỗi mẫu BR3, vì số thứ tự lần nộp được tính bằng cách quét toàn bộ bài nộp của người học.

Bảng 5.5. Latency BR1–BR3 (ms)

Đường đo	n	Median	p90	Nhỏ nhất	Lớn nhất
BR1: sự kiện → thông báo đọc được qua API	100	12,4	14,5	8,3	60,9
BR2: API gợi ý có feature store	150	2,2	4,2	1,4	12,2
BR3: nộp bài → 202	100	14,2	17,4	12,2	25,5
BR3: nộp bài → `graded`	100	71,0	77,3	65,4	90,2

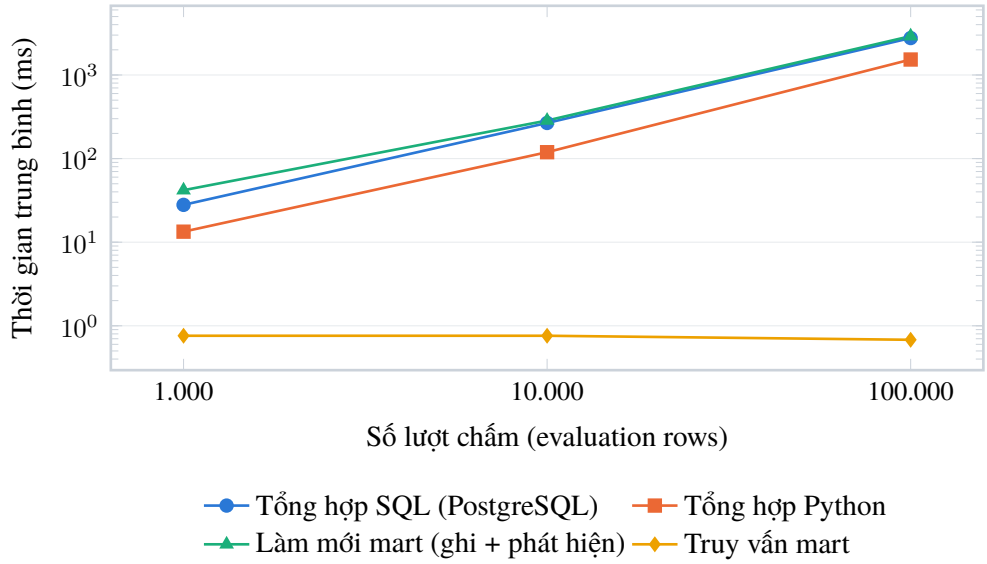
BR1 đạt median khoảng 12 ms, phù hợp với yêu cầu thông báo gần thời gian thực. BR2 có median khoảng 2,2 ms và p90 4,2 ms; 148/150 request nằm dưới ngưỡng 10 ms của yêu cầu tra cứu feature, hai request còn lại mất 10,5 và 12,2 ms. BR3 nhận 202 sau khoảng 14 ms và hoàn tất chấm sau khoảng 0,07 giây; sau khi trừ 14 ms nhận 202, phần còn lại chủ yếu là thời gian chạy sandbox và các chặng Kafka. Năm lượt đo cho kết quả ổn định, không trôi dần: median BR1 12,1–13,6 ms, BR2 1,8–4,1 ms, BR3 13,7–14,4 ms và 70–72 ms; p90 chỉ cao hơn median 2–6 ms. Hai giá trị lớn nhất của BR1 (54,8 và 60,9 ms) là các mẫu đơn lẻ.

5.4.2 BR4: tổng hợp SQL so với Python

Hai cách dùng cùng tập dữ kiện hợp lệ và cùng ngữ nghĩa chỉ số; kết quả khớp ở mọi kích thước và lần lặp. Hình 5.1 cho thấy thời gian tăng gần tuyến tính theo số lượt chấm. Python nhanh hơn khoảng hai lần (1,8–2,2 lần theo trung bình) nhờ dùng lại dữ kiện đã join cho cả hai phép tổng hợp, còn SQL phải lặp lại một số phép join JSONB. Ở 100.000 lượt chấm, một trong ba lần lặp của Python chậm hẳn (1,79 giây so với 1,38 và 1,42 giây), nên trung bình 1,53 giây cao hơn median 1,42 giây; tính theo median thì tỉ lệ ở kích thước này là 2,0 lần. Phép tổng hợp Python chỉ là đối chứng: trên đường sản phẩm, nhóm vẫn tổng hợp bằng SQL và công bố mart trong một transaction PostgreSQL vì ngữ nghĩa rõ ràng; truy vấn mart đã tổng hợp mất khoảng 0,7–0,8 ms.

Phép đo này được chạy lại trên mã sau khi gộp (commit `05bb5af`, 23/30 container, Bảng 5.3) và chỉ đo đường PostgreSQL: "tổng hợp SQL" là hai truy vấn SQL của PostgreSQL trên `anal`

`ytics.clean_rows` (JSONB), còn "làm mới mart" là hàm `refresh` của PostgreSQL, nay kết thúc bằng bước công bố có advisory lock và quality gate trên từng dòng. Trên mã đã gộp, engine tổng hợp mặc định của BR4 là Spark trên Delta (`EDU_BR45_ASSESSMENT_ENGINE=spark`), còn đường PostgreSQL là engine đối chứng; benchmark này không đo engine Spark.



Hình 5.1. BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log)

5.4.3 BR5: end-to-end latency và throughput của window

Lần chạy chấp nhận trên đường CDC thật mất khoảng **10.164 ms** từ khi ghi dữ liệu nguồn đến khi lưu kết quả rủi ro và cảnh báo. Khởi động lại Spark và phục hồi mất khoảng **20.142 ms**, không sinh cảnh báo trùng sau replay. Hai số này lấy từ lần chạy kịch bản chấp nhận trước đó (không đo lại). Thí nghiệm quy mô (Bảng 5.6) đo riêng chặng Kafka → Spark → window trong PostgreSQL với khoảng 1% bản ghi trùng; mọi kết quả khớp với phép tổng hợp batch độc lập bằng Python. Thí nghiệm này được đo lại trên mã sau khi gộp (commit `05bb5af`, 23/30 container, Bảng 5.3) với job br45-stream của mã đó: Spark 3.5.5 dùng state store RocksDB, và cùng driver còn chạy thêm truy vấn landing Delta của BR4.

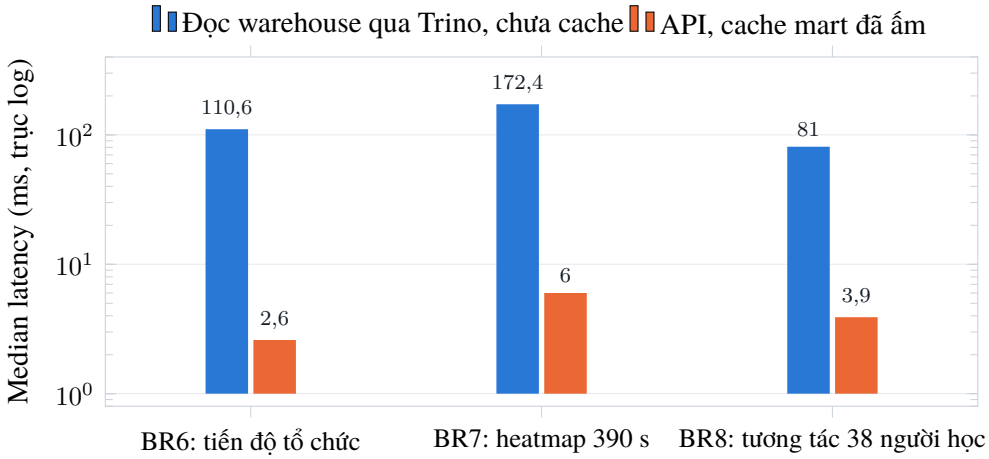
Bảng 5.6. BR5: chặng Kafka → Spark → window đã lưu (trung bình 3 lần)

Sự kiện duy nhất	Kafka đến window đã lưu	Throughput quan sát	Tính batch bằng Python
1.000	6.161 ms	170 sự kiện/s	1,8 ms
10.000	4.843 ms	2.088 sự kiện/s	18,4 ms
100.000	14.707 ms	6.800 sự kiện/s	150,2 ms

Ở 1.000 và 10.000 sự kiện, từng lần đo mất 4,1–8,2 giây và không tăng theo khối lượng: job

chạy micro-batch với trigger 5 giây, nên latency chủ yếu là thời gian chờ trigger. Tính batch rẻ hơn nhiều cho một snapshot tĩnh; streaming đổi lấy việc tiếp nhận liên tục, xử lý sửa đổi, replay, checkpoint và cập nhật tự động, trong khi batch theo lịch phải chờ lượt chạy mới thấy thay đổi. Nhóm không đưa ra số liệu false positive hay độ hiệu chỉnh của risk score vì không có nhãn dropout thật.

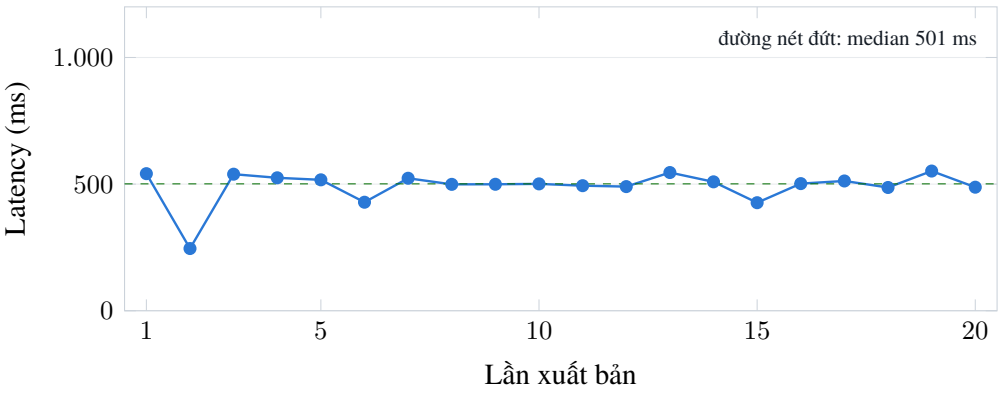
5.4.4 BR6–BR8: serving mart



Hình 5.2. Latency serving các mart BR6–BR8 (30 request mỗi trường hợp)

Bách phân vị 90 khi đọc trực tiếp qua Trino là 220, 218 và 111 ms cho BR6, BR7, BR8; khi có cache là 3,4, 7,4 và 5,5 ms. Vì mart chỉ đổi mỗi ngày còn dashboard thăm dò 5 giây/lần, bộ đọc dùng cache TTL 60 giây (`CachedMartReader`). Cache cũng khắc phục lỗi Trino hết bộ nhớ khi dashboard thăm dò liên tục.

5.4.5 BR8: từ xuất bản đến khi tìm kiếm được



Hình 5.3. BR8: latency từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp

API nhận yêu cầu xuất bản trong 31,6 ms (median). Toàn bộ đường đi PostgreSQL WAL → Debezium → Kafka → content-sync → tìm kiếm được có median **501 ms**, p90 541 ms và lớn nhất 552 ms. Theo log của content-sync trong lúc đo, bước đồng bộ chỉ mất 40–85 ms; phần còn lại là thời gian bắt thay đổi và chu kỳ thăm dò của consumer.

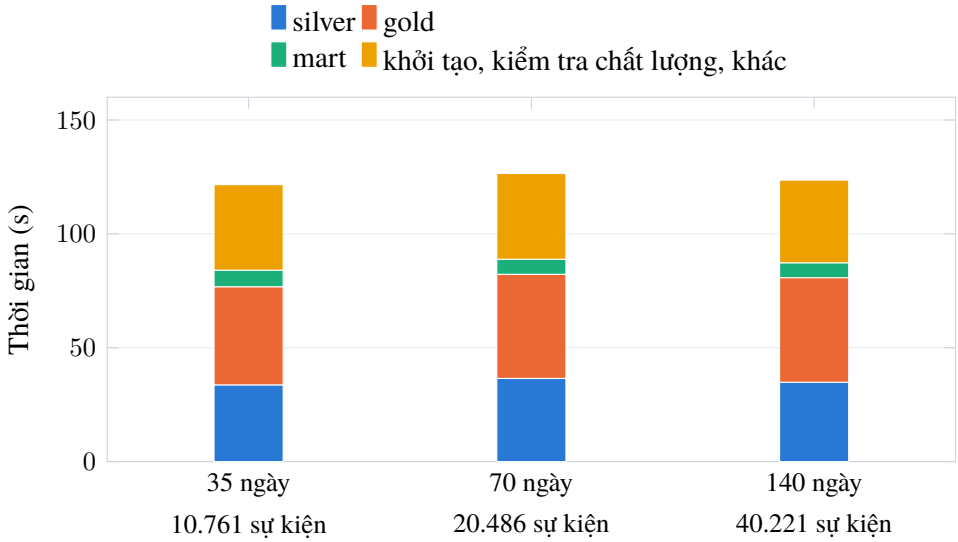
5.4.6 BR7/BR8: overlay thời gian thực

Bảng 5.7. Latency từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước)

Session mỗi đợt	Sự kiện thô (trung bình)	Trung bình (nhỏ nhất – lớn nhất)	Chính xác
25	234	5,71 s (4,76 – 6,93)	có
100	922	5,95 s (5,60 – 6,58)	có
400	3.618	6,13 s (5,36 – 7,16)	có

Latency gần như không đổi khi tải tăng 15 lần (trung bình tăng từ 5,7 lên 6,1 s) vì bị chi phối bởi chi phí cố định của các micro-batch nối tiếp (trigger 1 s; theo log của telemetry-live trong lúc đo, mỗi micro-batch kéo dài khoảng 0,8–6,4 s), chứ không phải bởi khối lượng. "Chính xác" nghĩa là overlay chứa đúng số tương tác đúng hạn của đợt cho video có nhiều tương tác nhất.

5.4.7 Thời gian build data warehouse



Hình 5.4. Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần)

Tổng thời gian build là 121,6; 126,5 và 123,6 giây cho 35, 70 và 140 ngày dữ liệu; mọi model đều thành công. Thời gian gần như phẳng khi khối lượng tăng 4 lần vì chi phí cố định của mỗi model (khởi tạo job Spark, commit Delta, kiểm tra chất lượng) chiếm phần lớn; mỗi mart BR6–BR8 chỉ mất 1,3–2,3 giây. Hai lần chạy cùng kích thước lệch nhau nhiều nhất 6,5 giây (123,3 và 129,8 giây ở 70 ngày). Mỗi lần build chạy trong một tiến trình Spark riêng, trên cấu hình 19 container của Bảng 5.3.

5.5 Debezium và polling: negative-control benchmark

Phần này không nhằm tìm một giải pháp nhanh hơn mà kiểm tra điều gì xảy ra khi thay Debezium bằng polling bảng nguồn theo `updated_at`. Đây là một negative control: polling hợp lý ở quy mô nhỏ nhưng cần chỉ ra điểm vỡ khi dữ liệu lớn.

5.5.1 Thiết lập

Bảng nguồn `bench.cdc_source` có 100.000 dòng, khoá chính, cột `updated_at` và index trên cột này (tạo bằng một lệnh `CREATE INDEX` sau khi nạp bảng, Phụ lục A). Debezium đọc WAL qua logical replication; polling chạy mỗi 0,5 giây với watermark `updated_at`. Sau snapshot, nhóm thực hiện một đợt gồm 100 UPDATE và 10 hard DELETE rồi đo số event nhận được, latency, số query và số tuple PostgreSQL trả về. Latency của polling được tính từ `updated_at` của dòng đến lúc poller đọc được; latency của Debezium được tính từ mốc `ts_ms` mà connector gắn vào event đến lúc consumer nhận được. Phép đo chạy ngày 13/11/2026 trên cấu hình 26 container của Bảng 5.3.

5.5.2 Kết quả

Bảng 5.8. Debezium và polling trên cùng 100.000 dòng

Chỉ số	Debezium	Polling
Event nhận được	110	100
UPDATE	100	100
Hard DELETE	**10**	**0**
Latency p50	485,9 ms	14,4 ms
Latency p95	487,2 ms	16,1 ms
Query bảng nguồn (SELECT/scan)	0	16
Query p50	–	18,6 ms
Tuple trả về do query bảng	0	4.061

Polling có index nên không full scan; đây là một so sánh công bằng. Tuy vậy polling vẫn thiếu 10 DELETE vì hard delete không còn dòng để `SELECT`; Debezium nhận đủ 100 UPDATE và 10 DELETE. Về latency, lần đo này không cho thấy Debezium nhanh hơn: polling đọc được 100 UPDATE sau median 14,4 ms, còn event của Debezium đến consumer sau median 485,9 ms.

Cần lưu ý: snapshot ban đầu của Debezium đọc 100.000 dòng nhưng nằm ngoài measurement window. Giá trị "0" trong bảng là 0 query bảng trong measurement window, không phải 0 SQL tuyệt đối; Debezium vẫn dùng replication protocol và metadata. Tương tự, `tup_returned` của polling là số tuple index/heap mà PostgreSQL trả về, không phải số event.

5.5.3 Ý nghĩa với bài toán dữ liệu lớn

Tốc độ. Phép đo này không chứng minh được Debezium nhanh hơn polling. Trong lần đo, polling có latency thấp hơn nhiều (p50 14,4 ms so với 485,9 ms của Debezium; p95 16,1 ms so với 487,2 ms). Cả hai giá trị đều nằm trong một chu kỳ 0,5 giây: polling thăm dò mỗi 0,5 giây và đợt UPDATE rơi ngay trước một lần thăm dò; latency gần 0,5 giây của Debezium phù hợp với `poll.interval.ms` mặc định 500 ms của connector, vốn không được chỉnh trong cấu hình đo. Vì chỉ có một đợt thay đổi, mỗi con số là một mẫu về vị trí của đợt thay đổi trong chu kỳ chứ không phải một phân phối latency; muốn so sánh tốc độ cần lặp nhiều đợt ở các pha khác nhau.

Tính đúng đắn. Polling chỉ thấy trạng thái hiện tại; hard DELETE biến mất khỏi bảng nên không còn gì để poll. Polling cũng có thể bỏ qua các UPDATE trung gian nếu một dòng bị sửa

nhiều lần giữa hai lần poll. Debezium giữ chuỗi thay đổi theo LSN, gồm cả DELETE và thứ tự giao dịch.

Stateful và retry. Debezium lưu offset/LSN và schema history trong Kafka Connect, nên restart có thể tiếp tục từ vị trí đã đọc. Polling không có trạng thái và phải tự lưu watermark; nếu poller chết giữa chừng hoặc commit watermark trước khi xử lý, event có thể bị mất hoặc bị lặp. Hard DELETE vẫn không thể phục hồi bằng watermark.

Scale dọc và ngang. Polling tăng tải theo số poller và số dòng/index phải quét; thêm poller là thêm query lên cùng bảng nguồn. Debezium đọc WAL với tải nguồn thấp và mở rộng theo partition/topic, consumer; nguồn không phải scan lại bảng khi dữ liệu tăng.

Ở 100.000 dòng, polling còn dùng được; ở 1–10 triệu dòng, chi phí index scan, số tuple trả về và số poller tăng tuyến tính trong khi Debezium vẫn đọc log. Với BR4–BR5 cần bắt sửa điểm, xoá và giữ thứ tự LSN, polling không bảo đảm đúng ngữ nghĩa dữ liệu; Debezium phù hợp hơn vì tính đúng đắn chứ không phải vì tốc độ.

5.5.4 Giới hạn của phép đo

Đây là smoke test 100.000 dòng trên một máy, chưa phải 1–10 triệu dòng. Nhóm chưa chạy failure injection cho connector. Latency của cả hai phương án chỉ được đo trên một đợt thay đổi nên phụ thuộc vào pha của chu kỳ thăm dò 0,5 giây và không dùng để xếp hạng tốc độ. Các số này đủ để chứng minh correctness, state và scale, nhưng không đủ để tuyên bố SLA production.

Riêng BR1–BR3, phép đo chạy trên cùng một máy đang gánh phần lớn stack (26/30 container) nên chịu ảnh hưởng của tranh chấp CPU và bộ nhớ. Hai đặc điểm của bản demo ảnh hưởng trực tiếp đến cách đo và là hạn chế cần xử lý: API inbox trả toàn bộ danh sách, nên inbox càng dài thì mỗi lần đọc càng chậm (152 thông báo: khoảng 68 ms, so với 7,5 ms khi còn 10 thông báo); cần phân trang. Thứ hai, số thứ tự lần nộp được tính bằng cách quét toàn bộ bài nộp của người học, nên chi phí nhận 202 tăng theo lịch sử (398 bài nộp: khoảng 167 ms, so với 28 ms khi chưa có lịch sử). Vì vậy mỗi mẫu BR1/BR3 được đo trên trạng thái rỗng. Hai cặp số chẩn đoán trên lấy từ lần đo trước trên máy khác (không đo lại).

5.6 Thảo luận

- **Đáp ứng yêu cầu latency.** Cảnh báo BR5 khoảng 10 giây (kịch bản chấp nhận, không đo lại) và tìm kiếm BR8 khoảng nửa giây đều đủ cho nghiệp vụ (cố vấn can thiệp, người học tìm nội dung mới). Overlay khoảng 6 giây phù hợp với dashboard trên mart theo ngày; muốn dưới một giây sẽ cần Flink (ADR 0014). Riêng ngưỡng 10 ms của BR2 đạt ở median

và p90, nhưng 2/150 request vượt ngưỡng.

- **Tách tính toán khỏi serving.** Mart được tổng hợp sẵn và cache giúp latency serving chỉ còn vài mili-giây dù truy vấn gốc qua Trino mất khoảng 80–170 mili-giây; các tra cứu theo key vẫn nằm ở Redis.
- **Đúng đắn quan trọng hơn tốc độ.** Ở BR4, nhóm giữ SQL dù Python nhanh hơn trên đường PostgreSQL và ghi rõ kết quả benchmark thay vì tuyên bố SQL thắng; hướng tối ưu (bảng có kiểu, dùng chung kết quả trung gian) đã được hiện thực trong engine mặc định Spark SQL trên Delta, nhưng engine này chưa có trong các phép đo của chương.
- **Lỗi phát hiện nhờ kiểm chứng.** Quá trình đo và kiểm thử đã phát hiện và sửa nhiều lỗi thật, ví dụ: bộ chunk bỏ mất khoảng 2/3 transcript; đồng hồ thiết bị chạy nhanh làm rơi sự kiện đúng hạn của người khác; cache tài liệu bị đọc trước khi kiểm tra phạm vi; Trino hết bộ nhớ khi dashboard thăm dò liên tục.

Chương 6

Hạn chế, hướng phát triển và kết luận

Chương này trình bày những hạn chế còn lại, hướng phát triển và kết luận, nhằm phân biệt rõ phần đã hiện thực với phần vẫn còn là giới hạn hoặc mới dừng ở mức thiết kế.

6.1 Hạn chế hiện tại

1. **Streaming latency ở mức giây.** Cảnh báo BR5 sau khoảng 10 giây (không đo lại) và overlay sau khoảng 6 giây, cao hơn mức dưới một giây mà Flink hướng tới. Đây là đánh đổi có chủ đích (ADR 0009, 0014).
2. **Embedding mang tính từ vựng.** Bộ nhúng băm không hiểu ngữ nghĩa ("join" không khớp "merge"); port `Embedder` và đường reindex cho phép thay bằng model thật.
3. **Overlay không mang phiên bản nội dung.** Khi video được quay lại, overlay trộn hai phiên bản nên chỉ hiển thị cho phiên bản mới nhất.
4. **Lịch sử BR5 trong data warehouse vẫn là dữ liệu tổng hợp.** Runtime BR5 nay ghi mỗi kết quả chấm điểm mới, mỗi cảnh báo mới và mỗi lần chuyển trạng thái cảnh báo vào outbox `analytics.domain_outbox` rồi phát lên `risk.learner_score.v1`, `risk.dropout_alert.v1` và `risk.alert_transition.v1`; landing của lakehouse nhận các topic `risk.*` và data warehouse có model silver cho cả ba. Tuy vậy các lần build warehouse trong báo cáo vẫn chạy trên dữ liệu sinh tổng hợp, chưa nạp sự kiện từ runtime thật, và bảng `silver.dropout_alert_transitions` chưa được model gold nào dùng.
5. **Hiệu năng BR4.** Mỗi lần sửa vẫn tính lại toàn bộ bản tổng hợp 30 ngày của khoá học bị ảnh hưởng thay vì tính incremental. Với 100.000 lượt chấm, một lần sửa trên engine đối chứng PostgreSQL (dùng phép join JSONB) tốn khoảng 2,9 giây; engine mặc định Spark SQL trên Delta chạy bất đồng bộ và chưa được đo ở quy mô này trong báo cáo.
6. **Vận hành đơn nút, bảo mật cục bộ.** Trino, OpenSearch và Kafka chạy một nút; plugin

bảo mật của OpenSearch bị tắt; nhân sự đăng nhập bằng tài khoản cấu hình tĩnh (mật khẩu băm Argon2id, session giữ trong bộ nhớ của một tiến trình API) còn script dùng token tĩnh; cả hai thay cho hệ thống định danh của trường; chưa có giám sát consumer latency, checkpoint time hay data freshness SLA.

7. **Dữ liệu tổng hợp, không có nhãn thật.** Thiếu nhãn dropout nên chưa đánh giá được độ chính xác của cảnh báo; các số đo hiệu năng chỉ là quan sát trên một máy.
8. **Lệch dữ liệu mẫu.** Seed lakehouse khai báo khoảng 18 khoá học và video không có trong LMS mẫu, nên job reindex báo lỗi.

6.2 Hướng phát triển

- Đưa stack đầy đủ thành mặc định khi triển khai; bổ sung retry và circuit breaker cho việc gửi thông báo.
- Chuyển các job cần latency dưới một giây sang Flink khi nền tảng lên Flink 2.x và connector Delta Kernel ổn định.
- Thay bộ nhúng băm bằng sentence-embedding và chạy một lần reindex.
- Chuyển các bảng snapshot (`fct_enrollment`, `fct_lesson_progress`) sang MERGE dựa trên Change Data Feed khi khối lượng đủ lớn; với dashboard cần độ tươi dưới giây ở quy mô lớn, bổ sung một engine OLAP chuyên dụng đọc qua Hive Metastore.
- Triển khai trên cụm nhiều nút với TLS và xác thực tập trung; bổ sung monitoring tập trung và quy trình xoay key pseudonym.
- Thu thập nhãn kết quả thật để hiệu chỉnh và đánh giá risk score của BR5.

6.3 Kết luận

Báo cáo đã trình bày Project, một nền tảng data engineering biến bốn nguồn dữ liệu rời rạc của hệ thống giáo dục trực tuyến thành data warehouse phục vụ tám yêu cầu nghiệp vụ.

Về kiến trúc, nền tảng bám sát kiến trúc tham chiếu: Kafka và Debezium cho ingestion, Spark cho xử lý, Delta Lake cho lakehouse, Redis cho serving latency thấp; lớp gold là data warehouse Kimball được phục vụ qua Trino. Các điều chỉnh — đáng kể nhất là dùng Spark Structured Streaming thay Flink cho BR5 và overlay — đều được ghi bằng ADR kèm lý do và hệ quả. Cách tổ chức DDD và kiến trúc hexagonal cho phép cùng một logic nghiệp vụ chạy trên nhiều engine và được kiểm thử độc lập.

Về hiện thực, BR4–BR8 chạy trên hạ tầng thật với các tính chất cốt lõi: xử lý theo event time,

chịu được late data, dữ liệu trùng lặp và out-of-order, rerun idempotent, pseudonymisation ở silver, k-anonymity cho báo cáo tổ chức và quality gate sau mỗi lớp. BR1–BR3 chạy end-to-end trên Kafka, Redis, PostgreSQL, Delta và Spark: sự kiện đi qua các dịch vụ độc lập, kết quả đến giao diện qua SSE, mọi đường ghi đều idempotent; BR2 có đủ hai nhánh feature. Giao diện web đưa kết quả đến từng nhóm người dùng.

Về đánh giá, hệ thống có 379 unit test, 1 integration test end-to-end, 12 browser test và các kịch bản nghiệp vụ đã được kiểm chứng tới tận giao diện. Số đo cho thấy latency phù hợp với từng yêu cầu: cảnh báo dropout khoảng 10 giây sau thay đổi nguồn (kịch bản chấp nhận, không đo lại), nội dung mới được index sau khoảng nửa giây, mart phục vụ trong vài mili-giây khi có cache.

Bài học lớn nhất là giá trị của việc kiểm chứng bằng dữ liệu và số đo: nhiều lỗi quan trọng chỉ lộ ra khi chạy hệ thống thật. Việc ghi rõ phần đã hiện thực và phần còn là thiết kế giúp đánh giá đúng mức độ hoàn thiện và định hướng phát triển.

Tài liệu tham khảo

[1] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[2] Advanced Distributed Learning Initiative. Experience API (xAPI) Specification. `https://github.com/adlnet/xAPI-Spec`, 2017.

[3] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*, 2011.

[4] Debezium Community. Debezium: Stream changes from your database. `https://debezium.io/documentation/`, 2026. Truy cập 11/2026.

[5] Matei Zaharia, Reynold S. Xin, Patrick Wendell, et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11):56–65, 2016.

[6] Michael Armbrust, Tathagata Das, Joseph Torres, et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[7] Michael Armbrust, Tathagata Das, Liwen Sun, et al. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.

[8] Raghav Sethi, Martin Traverso, Dain Sundstrom, et al. Presto: SQL on Everything. In *IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813, 2019.

[9] Redis Ltd. Redis Documentation. `https://redis.io/docs/`, 2026. Truy cập 11/2026.

[10] OpenSearch Project. OpenSearch Documentation: k-NN and hybrid search. `https://opensearch.org/docs/latest/`, 2026. Truy cập 11/2026.

[11] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[12] Alistair Cockburn. Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/, 2005.

[13] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[14] Michael Nygard. Documenting Architecture Decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions, 2011.

[15] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4), 2015.

[16] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.

[17] Databricks. What is the medallion lakehouse architecture? https://docs.databricks.com/en/lakehouse/medallion.html, 2026. Truy cập 11/2026.

[18] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.

[19] Latanya Sweeney. k-Anonymity: A Model for Protecting Privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.

[20] Yu. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[21] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.

[22] Instructure. Canvas LMS Guides. https://community.canvaslms.com/t5/Canvas-Guides/ct-p/canvas_guides, 2026. Truy cập 11/2026.

[23] Moodle HQ. Moodle User Documentation. https://docs.moodle.org/, 2026. Truy cập 11/2026.

[24] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/, 2018.

Phụ lục A

Hướng dẫn chạy và tái lập kết quả

A.1 Yêu cầu

Docker với 8 GB RAM cho VM: đủ để chạy từng nhóm BR, nhưng không đủ khi chạy đồng thời cả bốn profile (xem Mục 5.3); Python 3.11+; Node.js 20+. PostgreSQL chạy mặc định ở cổng 5433 để tránh xung đột.

A.2 Khởi động

```
# BR1/BR3: full stack with Kafka, Redis, PostgreSQL and separate services
docker compose -f docker-compose.yml -f docker-compose.br13.yml up -d --build

# BR4/BR5: real CDC + Spark Structured Streaming
docker compose --profile br45 up -d --build demo br45-init br45-cdc br45-
    stream
docker compose exec -T br45-cdc python -m edu_platform.br45.cli seed

# BR6/BR7/BR8: warehouse, content sync, live overlays
docker compose --profile br678 up -d --build     # first run takes about 4-5
    minutes
docker compose exec demo edu telemetry simulate --sessions 50

# BR2: live session branch + materialise long-term features into Redis
docker compose --profile br678 run --rm --no-deps telemetry-live edu stream
    telemetry-live --once
docker compose run --rm spark-jobs edu batch features-online --run-date
    2026-11-11
```

```
# Warehouse with synthetic data
docker compose run --rm spark-jobs edu warehouse bootstrap
docker compose run --rm spark-jobs edu warehouse seed-bronze --end-date
    2026-10-17 --days 35
docker compose run --rm spark-jobs edu warehouse run --run-date 2026-10-17 --
    start-date 2026-08-01
docker compose exec trino trino --user analyst --catalog lakehouse
```

Mở ứng dụng tại `http://localhost:8000`. Overlay `br13` dựng các dịch vụ notification và grading trên Kafka, Redis, PostgreSQL.

Ba workspace Instructor, Advisor và Org admin yêu cầu đăng nhập bằng tài khoản nhân sự (`docs/STAFF_AUTH.md`). Compose mặc định không khai báo tài khoản nào (`EDU_STAFF_ACCOUNTS` rỗng) và chỉ khai báo một token Bearer của người học (`learner-demo`). Vì vậy cần tạo tài khoản và khai báo token cho script trong shell *trước khi* chạy các lệnh `docker compose ... up` ở trên; nếu `demo` đang chạy thì chạy lại lệnh `up` đó để tạo lại container với `EDU_STAFF_ACCOUNTS` và `EDU_BR45_AUTH`.

```
# Staff accounts: each command asks for a password (12-128 characters)
# and prints one account with its Argon2id hash
umask 077; accounts_dir=$(mktemp -d)
.venv/bin/python -m edu_platform.br45.auth instructor instructor.demo \
  instructor-local "Demo Instructor" --course c-py-201 --course c-sql-101 \
  --course c-de-301 > "$accounts_dir/instructor.json"
.venv/bin/python -m edu_platform.br45.auth advisor advisor.demo \
  advisor-local "Demo Advisor" --course c-py-201 \
  > "$accounts_dir/advisor.json"
.venv/bin/python -m edu_platform.br45.auth admin admin.demo admin-local \
  "Demo Admin" --organisation org-acme --organisation org-globex \
  > "$accounts_dir/admin.json"
.venv/bin/python -m edu_platform.br45.auth admin acme-admin.demo \
  acme-admin-local "Acme Admin" --organisation org-acme \
  > "$accounts_dir/acme-admin.json"
export EDU_STAFF_ACCOUNTS="$(jq -cs 'add' "$accounts_dir"/*.json)"

# Bearer tokens for scripts (e2e check, BR6-BR8 benchmarks, Playwright API
# checks). Local demonstration values only: token -> identity and scope.
export EDU_BR45_AUTH='{
  "instructor-demo": {"id": "instructor-local", "role": "instructor",
    "courses": ["c-py-201", "c-sql-101", "c-de-301"]},
  "advisor-demo": {"id": "advisor-local", "role": "advisor",
    "courses": ["c-py-201"]},
  "admin-demo": {"id": "admin-local", "role": "admin",
    "organisations": ["org-acme", "org-globex"]},
```

```
  "acme-admin-demo": {"id": "acme-admin-local", "role": "admin",
    "organisations": ["org-acme"]},
  "learner-demo": {"id": "br45-demo-at-risk", "role": "learner",
    "courses": ["c-py-201"]}}'
```

Đăng nhập trên giao diện bằng tên người dùng vừa tạo (`instructor.demo`, `advisor.demo`, `admin.demo` cho cả hai tổ chức, `acme-admin.demo` chỉ cho Acme) và mật khẩu đã nhập; session có hạn tám giờ và mất khi API khởi động lại.

A.3 Kiểm thử

```
.venv/bin/ruff check src tests && .venv/bin/mypy
.venv/bin/python -m pytest tests/unit                          # 379 unit tests
EDU_BR13_BASE_URL=http://localhost:8000 .venv/bin/python -m pytest -m
    integration tests/integration/test_br13_services.py
# 11 Spark tests; require JDK, pyspark and delta-spark (the two BR4/BR5 ones
# are skipped unless EDU_BR45_RUN_SPARK_TESTS=1)
.venv/bin/python -m pytest -m spark tests/integration
# BR4/BR5 e2e: both tokens must be in EDU_BR45_AUTH of the demo service
BR45_VERIFY_INSTRUCTOR_TOKEN=instructor-demo \
  BR45_VERIFY_ADVISOR_TOKEN=advisor-demo \
  EDU_BR45_E2E=1 .venv/bin/python -m pytest tests/e2e/test_br45.py
cd apps/web && npm ci && npm run build && npx playwright install chromium
BR45_BASE_URL=http://localhost:8000 npx playwright test tests/br123.spec.ts
     # BR1--BR3
# All 12 tests: staff sign-in uses the password chosen above, the API
# checks use the Bearer tokens; the classroom, replay and staff-session
# tests are skipped unless their own BR45_BROWSER_* variables are set
BR45_BROWSER_STAFF_PASSWORD='<staff password>' \
  BR45_VERIFY_INSTRUCTOR_TOKEN=instructor-demo \
  BR45_VERIFY_ADVISOR_TOKEN=advisor-demo BR45_VERIFY_ADMIN_TOKEN=admin-demo \
  BR45_VERIFY_ACME_ADMIN_TOKEN=acme-admin-demo \
  BR45_BASE_URL=http://localhost:8000 npx playwright test
```

A.4 Đo hiệu năng

Các phép đo dùng stack như mục khởi động: hai tệp Compose với các profile `br45`, `br678`, `airflow` và `pipelines`, đúng như cấu hình mô tả ở Chương 2. Với Docker 8 GB, stack đầy đủ không chạy ổn định; trước mỗi phép đo cần tạm dừng (`docker compose stop`) các dịch vụ mà phép

đo không dùng, theo đúng cấu hình ở Bảng 5.3, và kiểm tra không có container nào khởi động lại trong lúc đo. Năm lượt BR1–BR3 được gộp thành `report/data/br123_latency.csv`; các tệp kết quả còn lại được sao chép vào `report/data/`. Trên mã đã gộp nhánh BR4/BR5, ba lệnh `edu benchmark serving`, `content` và `live` gọi API bằng hai token Bearer cố định trong mã (`admin-demo`, `instructor-demo`), nên dịch vụ `demo` phải chạy với `EDU_BR45_AUTH` như ở mục khởi động.

```
for i in 1 2 3 4 5; do                                  # BR1--BR3, 5 runs
  .venv/bin/python benchmarks/br123_latency.py \
    --samples 20 --br2-samples 30 \
    --out report/data/br123_latency_run$i.csv
done
.venv/bin/python -m edu_platform.br45.cli benchmark          # BR4
.venv/bin/python -m edu_platform.br45.stream_benchmark       # BR5
.venv/bin/edu benchmark serving --requests 30 \
  --out docs/evidence/br678/serving_latency.csv
.venv/bin/edu benchmark content \
  --out docs/evidence/br678/content_sync_latency.csv
.venv/bin/edu benchmark live --out docs/evidence/br678/live_overlay.csv
# Warehouse build: one process per (days, repeat) inside spark-jobs
docker compose run --rm -v "$PWD/docs/evidence/br678:/out" spark-jobs \
  edu benchmark marts --days 35 --repeat 1 --first-repeat 1 \
  --out /out/mart_build.csv     # repeat for 35/70/140 days, repeats 1-2
```

A.5 Đo Debezium và polling

Negative-control benchmark dùng bảng nguồn 100.000 dòng:

```
# Generate the facts CSV if missing (deterministic, seed 42)
.venv/bin/python benchmarks/cdc/generate_facts.py \
  --rows 1000000 --output benchmarks/cdc/data/org-facts-1m.csv

# Create the source table, then the updated_at index
# (setup_source does not create the index)
.venv/bin/python -m benchmarks.cdc.setup_source \
  benchmarks/cdc/data/org-facts-1m.csv --rows 100000
docker compose exec postgres psql -U lms -d lms -c \
  "CREATE INDEX cdc_source_updated_at_idx ON bench.cdc_source (updated_at)"

# Register the Debezium connector (once)
curl -sS -X DELETE http://localhost:8083/connectors/bench-cdc >/dev/null 2>&1
    || true
```

```
curl -sS -X POST -H 'Content-Type: application/json' \
  --data @benchmarks/cdc/connector.json http://localhost:8083/connectors

# Wait for the snapshot to reach 100,000 offsets
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --broker-list localhost:9092 --topic bench.bench.cdc_source

# Run 100 UPDATEs + 10 DELETEs
.venv/bin/python -m benchmarks.cdc.run_benchmark \
  --duration 8 --updates 100 --deletes 10 \
  --out benchmarks/cdc/results/cdc_100k_indexed.json
```

A.6 Biên dịch báo cáo

```
cd report
tectonic -X compile main.tex   # ặhoc: latexmk -xelatex main.tex
```

Trên Overleaf, chọn trình biên dịch XeLaTeX.

Phụ lục B

Tóm tắt API

Mọi endpoint có tiền tố `/api`; tài liệu OpenAPI ở `http://localhost:8000/docs` mô tả contract chi tiết. Các endpoint phân tích đều yêu cầu xác thực: trình duyệt đăng nhập bằng tài khoản nhân sự qua `/auth/staff/{role}/session` rồi gửi cookie session kèm header `X-Staff-Role`, còn script gửi token Bearer đã khai báo trong `EDU_BR45_AUTH`. Thiếu hoặc sai thông tin xác thực trả 401, sai vai trò hoặc ngoài phạm vi trả 403, không có dữ liệu trả 404, truy vấn sai trả 422, kho chưa sẵn sàng trả 503.

Bảng B.1. Các endpoint chính

BR	Phương thức	Đường dẫn	Mục đích
1	GET	`/learners/{id}/notifications`	Thông báo của người học
2	GET	`/learners/{id}/recommendations`	Gợi ý và khoá đã ghi danh
3	POST	`/submissions`	Nộp bài (202, xếp hàng)
3	GET	`/submissions/{id}`	Kết quả và phản hồi
4	GET	`/instructor/courses/{course}/questions/statistics`	Thống kê câu hỏi
4	GET	`/instructor/questions/{question}/test-cases`	Thống kê test case
5	GET	`/advisor/courses/{course}/at-risk-students`	Queue rủi ro, tóm tắt mức
5	GET	`/advisor/students/{student}/risk-history`	Lịch sử điểm
5	GET	`/advisor/alerts`	Danh sách cảnh báo

BR	Phương thức	Đường dẫn	Mục đích
5	PATCH	`/advisor/alerts/{id}/acknowledge`, `/resolve`	Alert lifecycle
6	GET	`/admin/organisations/{org}/skill-progress`	Báo cáo tổ chức (k = 5)
7	GET	`/instructor/courses/{course}/videos`	Video và phiên bản
7	GET	`/instructor/videos/{id}/interaction-heatmap`	Heatmap kèm overlay
8	GET	`/instructor/courses/{course}/engagement-trend`	Xu hướng tương tác
8	POST	`/instructor/content/{id}/versions`	Xuất bản phiên bản (202)
8	GET	`/instructor/courses/{course}/search`	Tìm kiếm lai hoặc từ khoá
–	POST, GET, DELETE	`/auth/staff/{role}/session`	Đăng nhập, đọc và huỷ session nhân sự
–	GET	`/events`	Server-Sent Events cho sự kiện trực tiếp